



eParking

Documentation technique

Ce document a pour but de décrire l'analyse, la conception, et la réalisation du projet eParking

Auteur Yoan Gilliand

Numéro du candidat 149380

Date de création 7 mai 2024

Dernière édition 23 mai 2024

Table des matières

1. Résumé du rapport.....	4
2. Préambule.....	5
2.1. Introduction	5
2.2. Organisation	5
2.3. Environnement de travail	5
2.4. Déroulement du projet.....	6
2.5. Déclarations	8
3. Cahier des charges	9
4. Analyse	10
4.1. État initial	10
4.2. État désiré	10
4.3. Public cible	10
4.4. Besoins.....	11
4.4.1. Cas d'utilisation	11
4.4.2. Description.....	12
4.5. Fonctionnement	15
4.5.1. Authentification.....	15
4.5.2. Formulaire.....	17
4.5.3. Administration.....	20
4.6. Ébauches conceptuelles	22
4.7. Stockage des données	26
4.8. Protection des données	26
4.9. Technologies utilisées.....	27
5. Conception	28
5.1. Schéma d'architecture.....	28
5.2. Diagrammes de séquences.....	29

5.3.	Base de données.....	37
5.4.	Mockups	43
5.5.	Stratégie de tests	46
6.	Réalisation.....	47
6.1.	Structure du projet.....	47
6.2.	Lint et code formatter.....	48
6.3.	Layout et design.....	49
6.4.	Gestion des configurations	49
6.5.	Authentification	50
6.6.	Validation des données du formulaire	56
6.7.	Gestion des fichiers.....	58
6.8.	Base de données.....	58
6.9.	Envoi d'emails	61
6.10.	Traitement des erreurs	63
7.	Tests de fonctionnement.....	66
8.	Problèmes rencontrés.....	67
9.	Amélioration et évolution	68
10.	Conclusions	69
11.	Remerciements	70
12.	Déclaration de non plagiat	70
13.	Sources et références	71
14.	Glossaire.....	73
15.	Annexes	74
16.	Table des illustrations.....	75

1. Résumé du rapport

Situation de départ :

Le projet eParking a été initié pour moderniser la gestion des demandes de vignettes de stationnement à la HES-SO//FR. Initialement, ce processus se faisait via des formulaires papier, causant des inefficacités et des erreurs de saisie. Les données devaient être ressaisies manuellement dans des tableurs, ce qui augmentait la charge administrative et les risques d'erreurs.

Mise en œuvre :

Pour répondre à ces défis, une plateforme web a été développée en utilisant Node.js, Express, et Vue.js, avec une base de données MySQL. Le système comprend un formulaire numérique pour la soumission des demandes, intégrant des validations automatiques et des sélections conditionnelles pour minimiser les erreurs de saisie. L'authentification des utilisateurs est assurée par la « CA TIC Authentication Gateway », ce qui garantit une sécurité renforcée grâce à l'authentification MFA de Microsoft. Une interface d'administration permet de gérer les demandes : les administrateurs peuvent consulter les détails, accepter ou refuser les requêtes, et exporter les données au format CSV. Les documents annexes, comme les copies de cartes grises, sont stockés de manière sécurisée sur le serveur.

Résultats :

La plateforme eParking améliore considérablement l'efficacité du processus de gestion des demandes. Le formulaire en ligne permet de réduire les erreurs de saisie et les délais de traitement. L'interface d'administration facilite la gestion des demandes, avec des fonctionnalités d'exportation et de notification par email automatisées, réduisant la charge de travail des administrateurs. L'intégration de l'authentification multi facteur a renforcé la sécurité du système, assurant que seules les personnes autorisées peuvent accéder aux fonctionnalités de gestion des vignettes. Enfin, la centralisation des données a permis une meilleure traçabilité et une gestion plus fluide des demandes.

2. Préambule

2.1. Introduction

Le projet « eParking » est un travail de CFC réalisé dans le cadre de la formation d'informaticien d'entreprise à la Haute école d'ingénierie et d'architecture de Fribourg. Il vise à développer une plateforme web pour la gestion des demandes de vignettes de parking de la HES-SO//FR. Ce document propose un suivi de l'avancement du projet.

2.2. Organisation

Candidat :	Yoan Gilliand, <u>HEIA-FR</u>
Chef de projet :	Joël Dacomo, HEIA-FR
Expert principal :	Jean-Pierre Gauch, <u>EMF</u>
Expert secondaire :	Nicolas Rouiller, EMF

2.3. Environnement de travail

Durant l'ensemble du travail, le matériel, les logiciels et les accès suivants sont utilisés :

A. Matériel

Les machines suivantes sont à disposition :

- MacBook Pro 16-inch 2019 (EIFRPOE01281), macOS Sonoma 14.3
- PC fixe (personnel), Windows 11 23H2

B. Logiciels et systèmes

Les logiciels et systèmes suivants sont utilisés :

- ChatGPT 4o
- Draw.io 24.3.1
- Dbdiagram.io 2024-03 release
- Figma Desktop 116.18.6
- Git 2.44.0
- Gitlab
- IntelliJ IDEA 2023.2.5 (Ultimate Edition)
- Lucidchart 2024-04 release
- Sequencediagram.org 9.12.2
- Suite Microsoft Office M365

C. Accès

Les accès suivants sont accordés :

- Réseau de la HEIA-FR
- Passerelle d'authentification Microsoft « CA TIC Authentication Gateway »

2.4. Déroulement du projet

A. Méthode de travail

Dans le cadre de ce projet, la méthode « waterfall » a été adaptée. Cette approche suit une progression linéaire, structurée en plusieurs phases: analyse, conception, réalisation et tests. Un ajout notable à l'approche traditionnelle est l'intégration de tests à la fin de chaque fonctionnalité développée plutôt qu'une phase de test unique en fin de projet. Cette adaptation permet de s'assurer que chaque fonctionnalité est pleinement opérationnelle avant de passer à la suivante, favorisant ainsi une meilleure qualité et stabilité du système tout au long du cycle de développement.

B. Sauvegarde des données

Les documents produits au cours de ce projet seront enregistrés chaque jour sur les disques réseau de la HEIA-FR, lesquels sont sauvegardés conformément à la politique de sauvegarde du service informatique de la HES-SO//Fribourg.

Source : <https://sinfo.hefr.ch/fr/catalogue-de-services/securite-et-protection-des-donnees/sauvegarde-des-donnees/>

De plus, un système de contrôle de version (VCS), ici Git, sera utilisé durant la phase d'implémentation pour assurer le suivi des différentes versions et l'évolution du projet.

C. Journal de bord

Dans le cadre de ce projet, un journal de bord détaillé est maintenu afin de pouvoir suivre de près les progrès, les défis et les résultats obtenus au quotidien. Ce journal de bord, qui couvre la période du début à la fin du projet, comporte des notes journalières incluant les travaux effectués, les objectifs atteints, les problèmes rencontrés et les solutions testées. Le journal de bord complété se trouve en annexe de ce document.

D. Planification

La planification est sous la forme d'un diagramme Gantt. Une planification initiale et une planification courante sont réalisées pour permettre de retracer le déroulement du projet. La superposition de ces deux dernières se trouve ci-dessous.

Dans la planification initiale, les tâches ont été découpées avec une granularité de 2 à 4 heures. Dans la planification courante, les tâches ont été découpées selon leur durée effective.

Légende : Les cases en couleurs représentent la planification initiale, les croix représentent la planification courante (temps effectif).

Tâches	Mardi 7 mai 24								Mercredi 8 mai 24								Lundi 13 mai 24								Mardi 14 mai 24								Mercredi 15 mai 24								
	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	
Administratif	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
Démarrage du projet	X	X							X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
Planification du projet			X	X																																					
Réunion avec le chef de projet						X																																			
1ère visite des experts							X																																		
2ème visite des experts																																									
Analyse					X	X	X	X	X	X	X	X	X	X	X	X																									
Analyse des besoins, des fonctionnalités					X				X	X	X																														
Ébauches conceptuelles													X	X																											
Conception																	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
Schémas UML																	X	X	X																						
Schéma de base de données																		X	X																						
Mockups																																									
Stratégie de tests																																									
Réalisation																																									
Mise en place (Git, Express, Vue)																																									
Création du formulaire d'envoi																																									
Enregistrement des demandes																																									
Affichage des demandes																																									
Affichage des détails d'une demande																																									
Acceptation ou rejet des demandes																																									
Export des demandes en CSV																																									
Authentification																																									
Tests de fonctionnement																																									
Tests de l'application selon la stratégie																																									
Documentation								X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
Documentation								X				X	X				X	X	X																						
Finalisation des documentations et envoi																																									

Figure 1 : Planification - Du 7 au 15 mai

Tâches	Jeudi 16 mai 24								Vendredi 17 mai 24								Mardi 21 mai 24								Mercredi 22 mai 24								Jeudi 23 mai 24								
	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	1	2	3	4	5	6	7	8	
Administratif																																									
Démarrage du projet																																									
Planification du projet																																									
Réunion avec le chef de projet																																									
1ère visite des experts																																									
2ème visite des experts																																									
Analyse																																									
Analyse des besoins, des fonctionnalités																																									
Ébauches conceptuelles																																									
Conception																																									
Schémas UML																																									
Schéma de base de données																																									
Mockups																																									
Stratégie de tests																																									
Réalisation	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
Mise en place (Git, Express, Vue)																																									
Création du formulaire d'envoi																																									
Enregistrement des demandes																																									
Affichage des demandes				X	X	X																																			
Affichage des détails d'une demande								X	X																																
Acceptation ou rejet des demandes													X	X																											
Export des demandes en CSV																	X	X																							
Authentification																																									
Tests de fonctionnement	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
Tests de l'application selon la stratégie	X						X			X				X				X							X		X	X													
Documentation	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
Documentation		X	X					X			X					X									X			X	X	X	X	X	X	X	X	X	X	X	X	X	
Finalisation des documentations et envoi																																							X	X	X

Figure 2 : Planification - Du 16 au 23 mai

Explications : La planification initiale a dans l'ensemble été respectée. Le temps gagné lors de l'analyse et la conception a pu être utilisé à bon escient pour la réalisation, ainsi que pour pouvoir faire des tests complets. Les heures supplémentaires restantes ont été utilisées pour la rédaction de la documentation.

E. Glossaire

À la fin de ce document se trouve un glossaire qui contient des explications sur les termes techniques et les abréviations utilisées. Les mots décrits dans ce glossaire sont soulignés et en italique lorsqu'ils apparaissent pour la première fois dans ce rapport.

(Exemple : L'exportation de la liste au format CSV)

2.5. Déclarations

A. Connaissances préalables

Le candidat possède les compétences techniques suivantes :

- Node.js
- Vue
- Express
- REST / RESTful
- MySQL / SQL
- Git

B. Travaux préparatoires

Bien qu'aucun travail préparatoire n'ait été effectué, il est pertinent de mentionner que le candidat a travaillé sur la « CA TIC Authentication Gateway », une passerelle d'authentification utilisant la MFA de Microsoft utilisée dans le projet eParking.

Plus d'informations : <https://auth-gateway.pages.forge.hefr.ch/ca-tic-authentication-gateway/>

C. Utilisation des modèles d'entreprise

Les modèles d'entreprises suivants sont utilisés :

- Documentation technique et journal de bord (intranet uniquement)
- Conventions de codages et de VCS : <https://api-ti.pages.forge.hefr.ch/>

D. Intelligence artificielle

Dans ce projet, l'intelligence artificielle est employée comme un outil d'assistance et non comme un substitut aux compétences humaines. Elle est utilisée pour aider à des tâches telles que la vérification des fautes d'orthographe, la suggestion de corrections grammaticales et l'amélioration de la clarté des textes.

3. Cahier des charges

Ce chapitre décrit la consigne du travail telle qu'elle a été communiquée au début du projet.

A. Donnée du problème

« La HES-SO//FR doit gérer les demandes de vignettes de stationnement pour les places de parking dans les zones dédiées. Chaque membre du personnel a le droit de faire une demande. Ces demandes peuvent être faites pour obtenir une autorisation (vignette autocollante) pour une année complète ou pour une période spécifique de 1 à 11 mois. Les demandes sont faites via un formulaire papier à remplir et à retourner à l'administration qui statue ensuite sur chaque demande. Toutes les demandes ne peuvent être acceptées dû au fait qu'il y a plus de demandes que d'autorisations disponibles.

Afin de simplifier le traitement des demandes, d'éviter de devoir ressaisir les données et de centraliser les documents à fournir, la HES-SO//FR désire se doter d'une plateforme Web. »

B. Description et objectifs du projet

« Réalisation d'un formulaire Web permettant de faire une demande. Ce formulaire devra permettre de saisir au minimum les informations du formulaire papier. Néanmoins, le candidat est responsable d'organiser une réunion avec le chef de projet afin de discuter des améliorations possibles à réaliser au début du projet. Le formulaire devra permettre d'uploader les documents annexes nécessaires tels que la copie de la carte grise du véhicule pour lequel la demande est faite.

Réalisation d'une partie Web de gestion des demandes. Un ensemble de personnes autorisées pourra accéder à la liste des demandes. Ces personnes pourront consulter l'ensemble des données ainsi que les fichiers joints. Il devra être possible d'exporter les données au format CSV. Depuis la liste des demandes, on pourra accepter ou refuser une demande ce qui déclenchera l'envoi d'un email de confirmation ou de refus à la personne ayant fait celle-ci.

L'ensemble des vues de l'application Web doivent être responsive (RWD). »

4. Analyse

4.1. État initial

La HES-SO//FR fait face à des difficultés dans la gestion des demandes de vignettes de stationnement, utilisées pour attribuer des places de parking dans les zones désignées de l'établissement. Actuellement, le processus implique l'utilisation d'un formulaire papier que chaque membre du personnel doit remplir et soumettre à l'administration pour traitement. Ce système manuel ralentit le traitement des demandes et crée également des goulots d'étranglement administratifs, surtout compte tenu du fait que le nombre de demandes excède souvent celui des vignettes disponibles.

Le problème est que la saisie se fait sur papier, ce qui est chronophage et sujet à des erreurs humaines lorsque les données sont ressaisies dans des tableurs afin d'être gérées. De plus, les informations sont dispersées et difficiles à suivre, rendant la gestion des états des demandes compliquée pour l'administration.

4.2. État désiré

Pour répondre à ces défis, la HES-SO//FR souhaite développer une plateforme Web qui automatisera et centralisera le processus de demande et de gestion des vignettes de stationnement. Ce système devra améliorer significativement l'efficacité du processus de sélection et de distribution des vignettes.

Un formulaire web remplacera le formulaire papier, permettant aux utilisateurs de soumettre leurs demandes en ligne. Ce formulaire inclura toutes les informations nécessaires qui étaient précédemment collectées manuellement.

Une interface de gestion permettra à un groupe désigné de personnel d'accéder et de gérer les demandes. Ils pourront voir toutes les données pertinentes, y compris les documents joints, et exporter ces données au format CSV.

Le système permettra aux gestionnaires de sélectionner facilement les candidats pour l'attribution des vignettes et d'envoyer automatiquement des confirmations ou des refus par email, améliorant ainsi la transparence du processus.



Figure 3 : Principe de fonctionnement

Explications : Les utilisateurs accèdent au formulaire et soumettent une demande de vignette. Celle-ci est stockée sur le serveur de l'application. Les administrateurs ont quant à eux accès à une interface de gestion des demandes, peuvent consulter les détails des demandes, les accepter, les refuser et les supprimer. Ils peuvent également exporter la liste en CSV pour l'ouvrir sur un tableur.

4.3. Public cible

L'application est destinée à l'ensemble des étudiants et des collaborateurs de la HES-SO//FR. En effet, toute personne désirant accéder aux zones de parking devra passer par eParking.

4.4. Besoins

4.4.1. Cas d'utilisation

Un diagramme de cas d'utilisation permet de représenter le comportement fonctionnel d'un système logiciel. Voici le diagramme de cas d'utilisation de l'application :

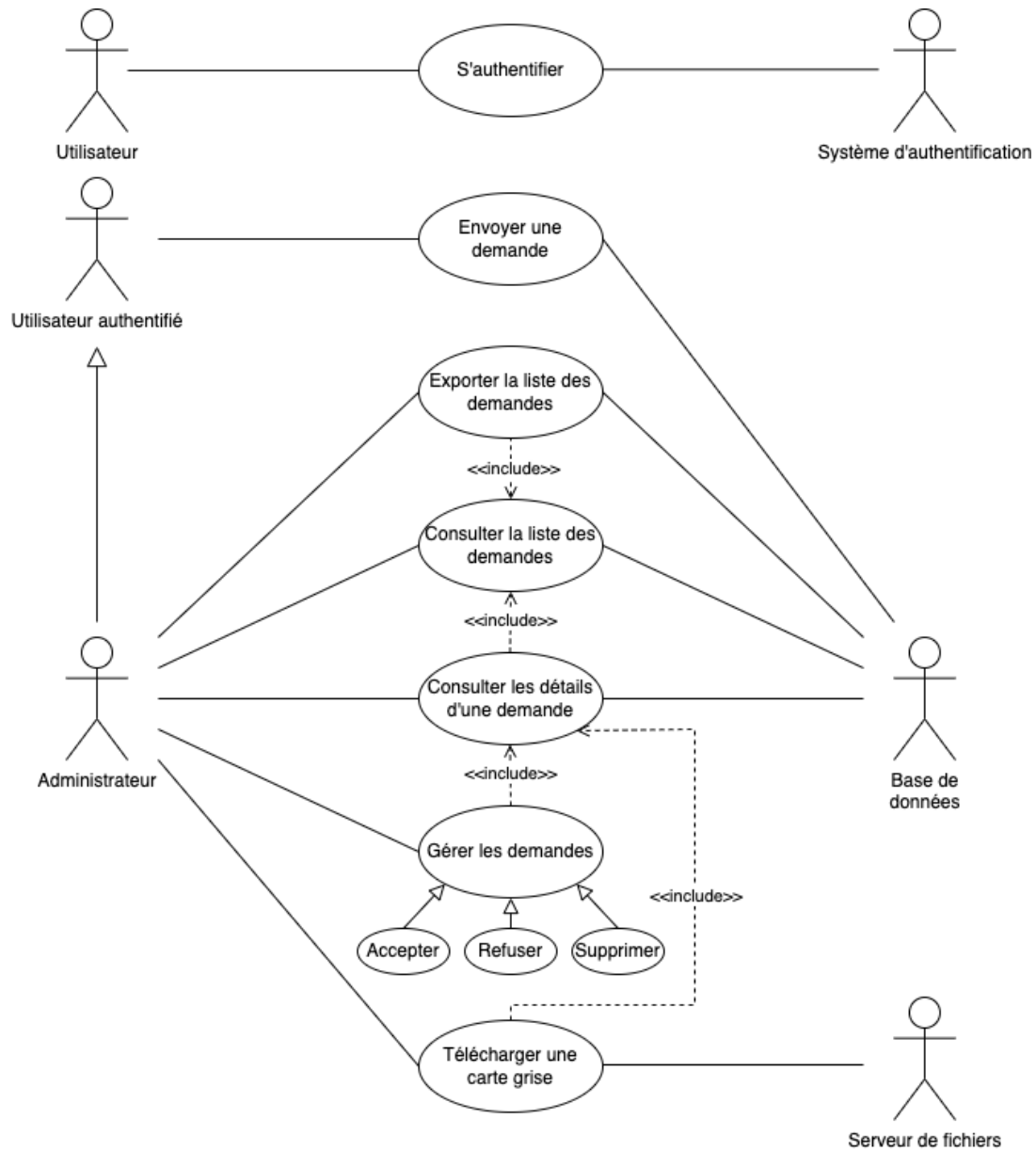


Figure 4 : Diagramme de cas d'utilisation

Explications :

La seule action que peut effectuer un utilisateur est de s'authentifier. Une fois authentifié, l'utilisateur aura la possibilité d'envoyer une demande. L'administrateur peut quant à lui accomplir les mêmes actions que l'utilisateur authentifié, mais il peut également consulter et exporter la liste des demandes, consulter les détails d'une demande, la gérer (l'accepter, la refuser ou la supprimer) et télécharger les cartes grises associées.

4.4.2. Description

Les besoins des utilisateurs de l'application proviennent du cahier des charges. Ceux-ci sont séparés en 2 groupes :

- Les besoins dits « communs », qui concernent tous les utilisateurs.
- Les besoins dits « administrateurs », qui concernent uniquement les utilisateurs authentifiés possédant un rôle « admin ».

A. Communs

La partie commune représente les actions disponibles à tout utilisateur possédant un compte autorisé. Tous les utilisateurs de la plateforme doivent s'authentifier pour accéder aux fonctionnalités. Après l'authentification, l'application sait quelles fonctionnalités afficher à quel utilisateur.

Authentification	
Quoi	Redirection de l'utilisateur sur une page de connexion
Pourquoi	Identifier les utilisateurs et éviter les demandes de personnes non autorisées
Contrainte	L'utilisateur doit posséder un login valide
Remarque	L'utilisateur doit ensuite être redirigé sur la page demandée

Envoyer une demande pour obtenir une vignette	
Quoi	Afficher un formulaire permettant de faire une demande de vignette
Pourquoi	Permettre aux utilisateurs d'obtenir une vignette pour leur véhicule
Comment	Formulaire web
Contrainte	Accessible uniquement par les utilisateurs authentifiés
Remarque	Le prénom et le nom du demandeur sont ceux de la personne authentifiée et ne peuvent être modifiés

B. Administrateurs

Les administrateurs ont accès à une interface dédiée qui leur permet de gérer efficacement les demandes de vignettes soumises par les utilisateurs de l'application. Ils peuvent consulter la liste complète des demandes, en visualiser les détails, les traiter en les acceptant ou en les refusant, et les exporter. Les administrateurs peuvent également supprimer les demandes obsolètes ou erronées.

Consulter la liste des demandes	
Quoi	Afficher une liste des demandes de vignettes
Pourquoi	Permettre aux administrateurs de visualiser toutes les demandes en cours ou traitées
Comment	Interface web

Exporter la liste des demandes	
Quoi	Exporter la liste des demandes au format CSV
Pourquoi	Permettre aux administrateurs d'analyser ou d'archiver les demandes
Comment	Interface web

Consulter les détails d'une demande	
Quoi	Afficher les détails complets d'une demande de vignette
Pourquoi	Permettre aux administrateurs d'évaluer correctement les demandes
Comment	Interface web

Télécharger les cartes grises	
Quoi	Télécharger les cartes grises associées aux demandes
Pourquoi	Permettre aux administrateurs de vérifier l'appartenance du véhicule
Comment	Interface web

Accepter ou refuser une demande	
Quoi	Permettre aux administrateurs d'accepter ou de refuser une demande de vignette
Pourquoi	Gérer efficacement les demandes et informer les utilisateurs de la décision
Comment	Interface web
Remarque	Un email doit être envoyé à l'utilisateur pour l'informer

Supprimer une demande	
Quoi	Permettre aux administrateurs de supprimer une demande de vignette
Pourquoi	Retirer les demandes obsolètes ou erronées
Comment	Interface web
Remarque	Une confirmation de suppression doit être demandée

C. Synthèse

Ci-dessous se trouve le tableau récapitulatif des besoins des utilisateurs en fonction de leur rôle.

Besoins	Utilisateurs authentifiés	Administrateurs
Authentification	X	X
Envoyer une demande pour obtenir une vignette	X	X
Consulter la liste des demandes		X
Exporter la liste des demandes		X
Consulter les détails d'une demande		X
Télécharger les cartes grises		X
Accepter ou refuser une demande		X
Supprimer une demande		X

4.5. Fonctionnement

Ce chapitre fournit des informations supplémentaires sur la mise en œuvre et le fonctionnement des différents besoins exprimés.

4.5.1. Authentification

A. Variantes sur la méthode d'authentification

Il y'a plusieurs méthodes d'authentification utilisables dans le cas du projet, chacune ayant ses avantages et ses inconvénients.

Variante	Description
Nom d'utilisateur et mot de passe unique pour chaque rôle	Avantages : - Simple à implémenter et à gérer - Rôles explicitement définis Inconvénients : - Difficile à faire évoluer - Gestion compliquée si les autorisations deviennent plus spécifiques - Risques de sécurité dus au fait que le mot de passe est public
Annuaire LDAP de la HES-SO//FR	Avantages : - Gestion centralisée des utilisateurs - Gestion plus simple des rôles et des permissions grâce aux groupes Inconvénients : - L'annuaire est utilisable seulement en production - L'annuaire est utilisable seulement depuis un réseau spécifique - Toutes les applications ne se voient pas accorder le droit à l'annuaire
CA TIC Authentication Gateway (Microsoft MFA)	Avantages : - Utilise le tenant Microsoft de la HES-SO - Gestion centralisée des utilisateurs, des rôles et des permissions - Authentification multifacteur Inconvénients : - Ne permet pas d'identifier spécifiquement les membres de la HES-SO//FR - Nécessite de spécifier les rôles manuellement dans une base de données

Conclusion :

Le choix se porte sur l'utilisation de la « CA TIC Authentication Gateway ». Cette décision s'explique par plusieurs raisons :

1. **Généralisation du MFA de Microsoft** : La CA TIC Authentication Gateway utilise le tenant Microsoft de la HES-SO, permettant ainsi la généralisation de l'authentification multifacteur (MFA) au sein de la HES-SO//FR pour une sécurité accrue.
2. **Double authentification plus sûre** : L'authentification multifacteur proposée par Microsoft renforce la sécurité en exigeant une deuxième méthode d'identification en plus du mot de passe.
3. **Restrictions du LDAP** : Les restrictions en cours sur l'annuaire LDAP rendent son accès de plus en plus limité, ce qui diminue les possibilités de l'utiliser.

Cependant, quelques remarques découlent de ce choix :

1. **Identification limitée** : Actuellement, la gateway ne permet pas de déterminer l'école spécifique de l'utilisateur se connectant, mais seulement son appartenance à la HES-SO. Pour remédier à cela, un autre moyen d'identification doit être mis en place.
2. **Gestion des rôles** : Les rôles doivent être spécifiés manuellement dans une base de données sur le serveur de la gateway. Par conséquent, seuls les administrateurs seront définis, tandis que les autres utilisateurs n'auront qu'à posséder un login valide sans attribution de rôle spécifique.

Cette solution apporte une gestion centralisée des utilisateurs tout en améliorant la sécurité grâce à l'authentification multifacteur, répondant ainsi aux besoins actuels du projet.

Lien : <https://auth-gateway.pages.forge.hefr.ch/ca-tic-authentication-gateway/>

B. Fonctionnement

Authentification	
Action	Description
Authentification d'un utilisateur	Redirection sur la page de connexion de la gateway. Une fois authentifié, l'utilisateur est redirigé vers la page initialement demandée.
Déconnexion d'un utilisateur	Redirection sur la page de déconnexion de la gateway. Une fois délogué, l'utilisateur est redirigé vers la page de connexion.

4.5.2. Formulaire

A. Champs du formulaire

Contrairement au formulaire papier, le formulaire numérisé offre une expérience utilisateur améliorée grâce à des validations automatiques, des champs conditionnels, et des sélections simplifiées via des menus déroulants ou des cases à cocher.

Chaque champ du formulaire possède donc un type ainsi que des restrictions :

Champ	Type	Restrictions
Informations personnelles		
Prénom	Texte	- Obligatoire - 50 caractères ou moins
Nom	Texte	- Obligatoire - 50 caractères ou moins
Email	Email	- Obligatoire - Format email - 254 caractères ou moins - Domaine <i>hefr.ch</i> ou <i>edu.hefr.ch</i>
Téléphone	Texte	- Obligatoire - 10 caractères
Adresse	Texte	- Obligatoire - 60 caractères ou moins
NPA	Numérique	- Obligatoire - Entre 1000 et 9999
Localité	Texte	- Obligatoire - 30 caractères ou moins
École	Sélecteur	- Obligatoire - Doit être une des écoles proposées
Filière	Sélecteur	- Obligatoire - Doit être une des filières proposées
Taux de travail	Numérique	- Obligatoire - Entre 1 et 100
Informations du véhicule		
Plaque d'immatriculation	Texte	- Obligatoire - 9 caractères ou moins - 2 lettres suivies de 1 à 7 chiffres

Carte grise	Fichier	- Obligatoire - Format PNG, JPG, JPEG, PDF
Autres occupants du véhicule		
Prénom	Texte	- Obligatoire - 50 caractères ou moins
Nom	Texte	- Obligatoire - 50 caractères ou moins
Adresse	Texte	- Obligatoire - 60 caractères ou moins
NPA	Numérique	- Obligatoire - Entre 1000 et 9999
Localité	Texte	- Obligatoire - 30 caractères ou moins
Période		
Période	Radio	- Obligatoire - Mensuelle ou annuelle
Début	Sélecteur	- Obligatoire - Ne peut pas être un mois passé ou le mois courant
Fin	Sélecteur	- Obligatoire - Ne peut pas être un mois passé ou le mois courant - Ne peut pas être avant le mois de début
Conditions générales		
Règles	Case à cocher	- Obligatoire
Protection des données	Case à cocher	- Obligatoire

Remarques

- Le choix de limiter les emails aux *hefr.ch* et *edu.hefr.ch* permet d'identifier l'école de l'utilisateur et donc de pallier le problème de la « CA TIC Authentication Gateway ».
- La case institut n'a plus lieu d'être car désormais, chaque membre d'un institut est contractuellement lié à une filière.
- Les limitations des caractères sont détaillées dans le chapitre « Base de données » de la conception.
- La liste des champs a été validée par le chef de projet.

Variantes sur la gestion des véhicules et des occupants

Un utilisateur peut avoir jusqu'à deux véhicules et plusieurs occupants liés à sa demande. Les approches sont les suivantes :

Variante	Description
Bouton pour ajouter ou supprimer	Avantages : - Interface épurée - Flexibilité Inconvénients : - Action supplémentaire
Affichage des formulaires dès le début	Avantages : - Plus rapide à remplir Inconvénients : - Interface encombrée - Non flexible

Conclusion

Pour la gestion des véhicules et des occupants, il a été décidé d'adopter l'option du bouton permettant d'ajouter ou de supprimer des formulaires. Ainsi, un seul formulaire de véhicule est affiché initialement, tandis qu'aucun formulaire d'occupant n'est visible au départ. Cette approche permet de ne pas encombrer l'interface et offre une flexibilité totale en permettant aux utilisateurs d'ajouter ou de supprimer des formulaires en fonction de leurs besoins.

Variantes sur les conditions générales

Les règles relatives aux vignettes et les conditions de protection des données ne peuvent pas être signées de manière manuscrite, contrairement au formulaire papier. Par conséquent, il est nécessaire d'ajouter des cases à cocher. Ces règles et conditions peuvent être consultées sur la page de l'intranet menant à l'application, ce qui évite d'avoir à les ajouter sur le site lui-même.

B. Fonctionnement

Vue du formulaire	
Action	Description
Affichage du formulaire	Afficher le formulaire. Ajouter des boutons pour ajouter et supprimer les véhicules et les occupants.
Envoi du formulaire	Ajouter un bouton qui est désactivé tant que tous les champs du formulaire ne sont pas correctement remplis.

4.5.3. Administration

Vue de la liste des demandes	
Action	Description
Affichage de la liste des demandes à traiter	Afficher les demandes sous forme de tableau. Les en-têtes des colonnes correspondent aux critères de sélection : - Nom - Prénom - École - NPA + Localité - Nombre d'occupants - Détails La colonne détails contient : - Un bouton pour consulter les détails d'une demande. Ajouter un bouton pour afficher toutes les demandes.
Affichage de la liste de toutes les demandes	Afficher les demandes sous forme de tableau. Les en-têtes des colonnes correspondent aux critères de sélection : - Nom - Prénom - École - NPA + Localité - Nombre d'occupants - État - Détails La colonne statut contient : - Un pictogramme de couleur selon le statut de la requête. La colonne détails contient : - Un bouton pour consulter les détails d'une demande. Ajouter un bouton pour afficher seulement les demandes non traitées.
Filtrage des demandes	Afficher une barre de recherche pour filtrer les demandes. Les en-têtes du tableau permettent de filtrer les demandes.
Exportation de la liste des demandes	Afficher un bouton permettant d'exporter la liste des demandes au format CSV.

Vue des détails d'une demande	
Action	Description
Affichage des détails de la demande	Afficher les informations de la demande (identifiant, date et statut) Afficher les données de l'utilisateur sous la forme du formulaire, avec les champs désactivés.
Téléchargement des pièces jointes	Afficher une icône à côté des champs « Carte grise » afin de pouvoir télécharger les pièces jointes.
Acceptation ou refus d'une demande	Ajouter deux boutons, un pour accepter et l'autre pour refuser une demande. L'acceptation ou le refus déclenche un message de confirmation et met à jour le statut. L'acceptation ou le refus déclenche l'envoi d'un email à la personne ayant envoyé la demande.
Suppression d'une demande	Ajouter un bouton pour supprimer une demande. Une fenêtre contextuelle demande une confirmation avant la suppression.
Retour à la liste des demandes	Ajouter un bouton pour retourner à la liste des demandes.

4.6. Ébauches conceptuelles

A. Authentification

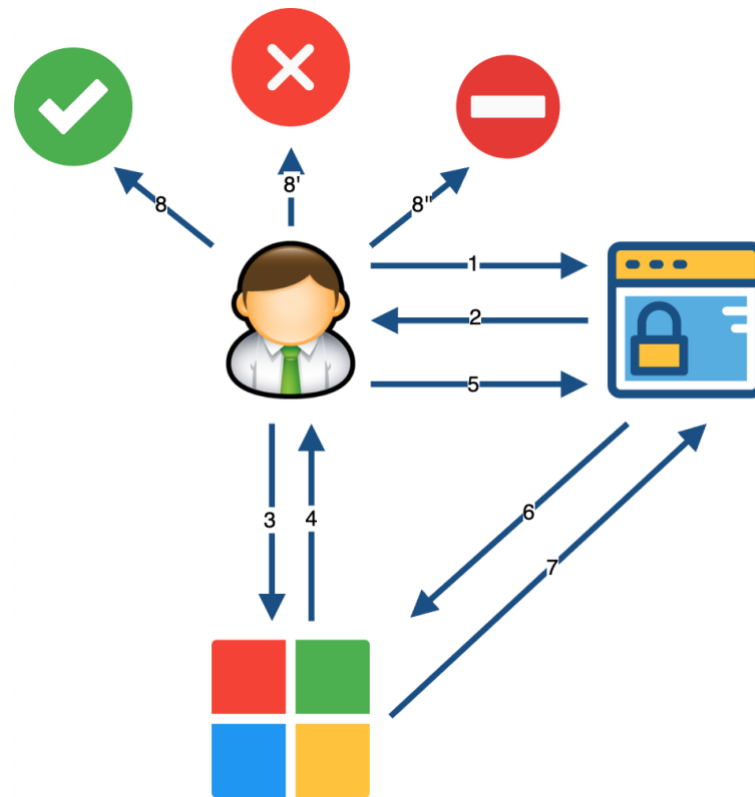


Figure 5 : Ébauche conceptuelle - Authentification

Explications :

1. L'utilisateur non-authentifié accède à une page de l'application
2. L'application redirige l'utilisateur vers la « CA TIC Authentication Gateway »
3. L'utilisateur se connecte à la « CA TIC Authentication Gateway » avec la MFA de Microsoft
4. La « CA TIC Authentication Gateway » redirige l'utilisateur vers la page demandée initialement et celui-ci obtient un jeton
5. L'utilisateur accède à nouveau à la page de l'application avec son jeton
6. L'application vérifie le jeton auprès de la « CA TIC Authentication Gateway »
7. La « CA TIC Authentication Gateway » indique à l'application la validité du jeton, ainsi que le rôle de l'utilisateur
8. Si le jeton est valide et que l'utilisateur possède les rôles nécessaires, il accède à la page
- 8'. Si le jeton n'est pas valide, l'utilisateur doit recommencer le processus
- 8''. Si le jeton est valide, mais que l'utilisateur n'a pas les droits nécessaires, il arrive sur une page d'erreur lui l'indiquant

Pour les prochains diagrammes, le processus de vérification du token et des rôles ne sera pas répété, mais il est identique.

B. Envoi d'une demande pour obtenir une vignette

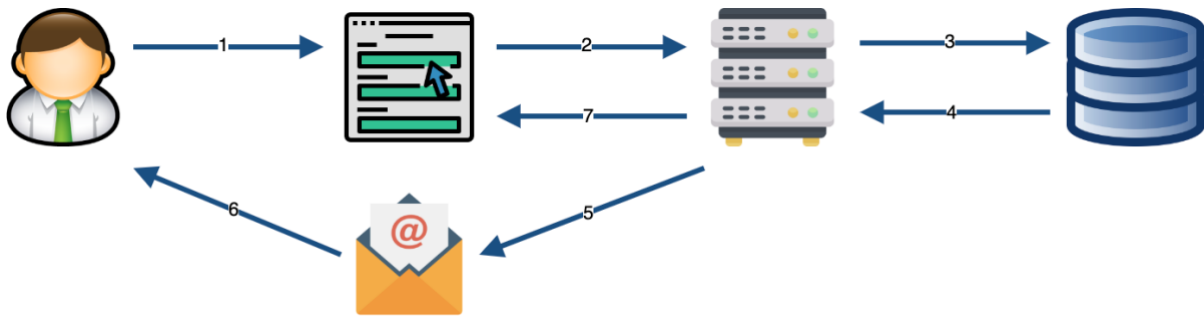


Figure 6 : Ébauche conceptuelle - Envoi d'une demande pour obtenir une vignette

Explications :

1. L'utilisateur remplit le formulaire
2. Le formulaire valide les données, et les envoie à au serveur web
3. Les données sont vérifiées et traitées, les fichiers sont enregistrés sur le serveur et les informations sont stockées dans la base de données.
4. La base de données confirme l'enregistrement
5. Le serveur web génère l'email de confirmation
6. L'email est envoyé à l'utilisateur
7. L'utilisateur reçoit une confirmation de l'envoi de la demande

C. Affichage de la liste des demandes



Figure 7 : Ébauche conceptuelle - Affichage de la liste des demandes

Explications :

1. L'administrateur accède à l'application
2. L'application récupère la liste des demandes depuis la base de données
3. Les demandes sont affichées

D. Téléchargement d'une carte grise



Figure 8 : Ébauche conceptuelle - Téléchargement d'une carte grise

Explications :

1. L'administrateur appuie sur le bouton pour télécharger une carte grise
2. L'application récupère la carte grise sur le serveur
3. La carte grise est retournée à l'application
4. La carte grise est téléchargée

E. Exportation de la liste des demandes

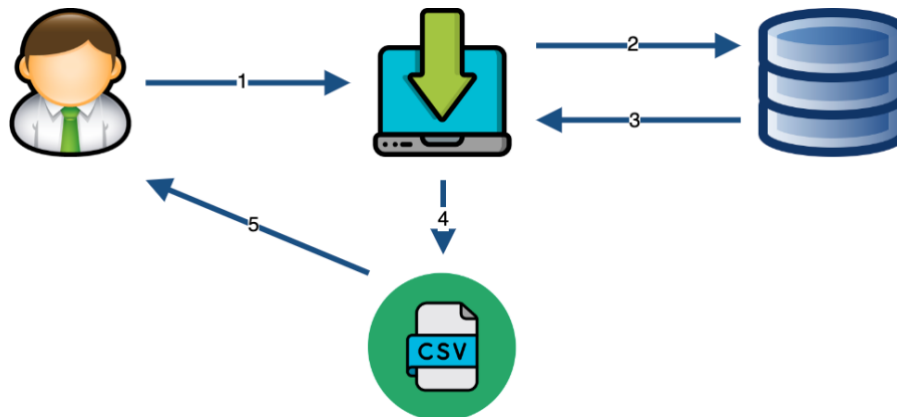


Figure 9 : Ébauche conceptuelle - Exportation de la liste des demandes

Explications

1. L'administrateur initie l'exportation
2. L'application récupère l'ensemble des détails des demandes
3. Les données sont retournées à l'application
4. Les données sont converties au format CSV
5. Le fichier CSV est retourné à l'administrateur

F. Affichage des détails d'une demande



Figure 10 : Ébauche conceptuelle - Affichage des détails d'une demande

Explications :

1. L'administrateur accède à l'application
2. L'application récupère les détails de la demande depuis la base de données
3. Les détails sont affichés

G. Acceptation ou refus d'une demande

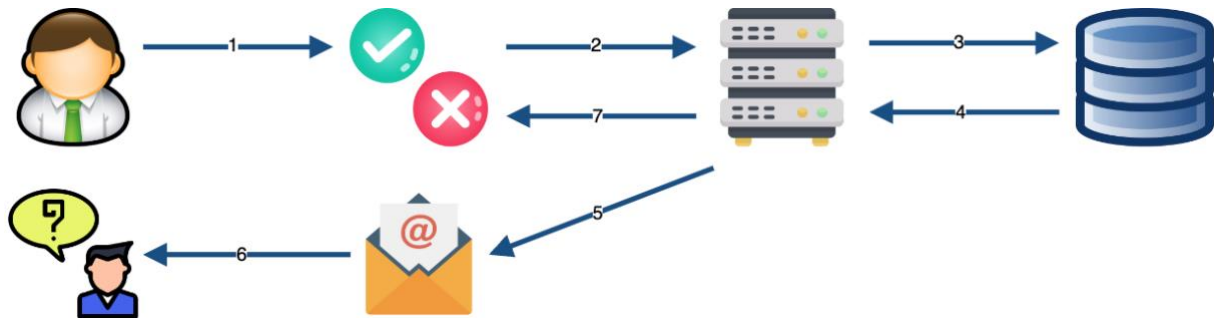


Figure 11 : Ébauche conceptuelle - Acceptation ou refus d'une demande

Explications :

1. L'administrateur accepte ou refuse la demande
2. Le serveur web reçoit la requête d'acceptation ou de refus
3. L'état de la demande est mis à jour dans la base de données
4. La base de données confirme la mise à jour
5. L'email pour l'utilisateur ayant fait la demande est généré
6. L'utilisateur ayant fait la demande reçoit l'email contenant la décision
7. L'administrateur reçoit une confirmation de l'acceptation ou du refus

H. Suppression d'une demande

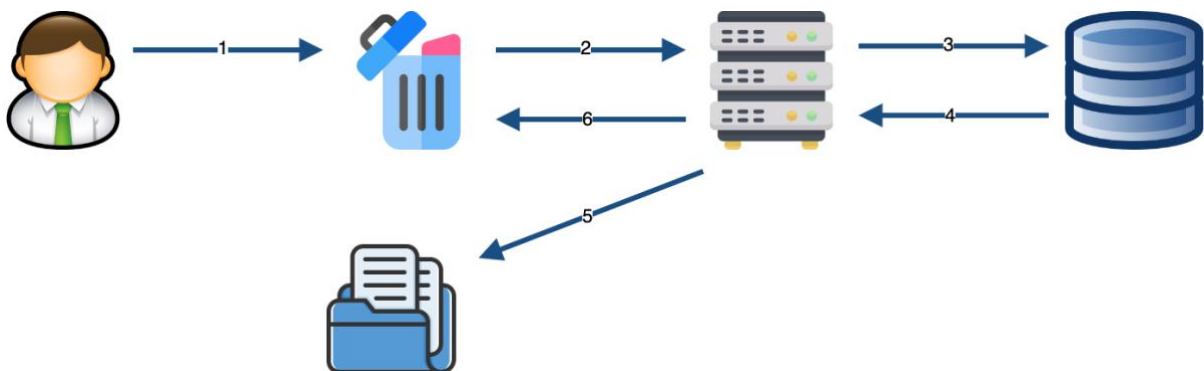


Figure 12 : Ébauche conceptuelle - Suppression d'une demande

Explications :

1. L'administrateur initie la suppression
2. Le serveur web reçoit la requête de suppression
3. Les données sont supprimées de la base de données
4. La base de données confirme la suppression
5. Les cartes grises sont supprimées
6. L'utilisateur reçoit une confirmation de la suppression de la demande

4.7. Stockage des données

Le principal besoin en matière de stockage concerne la sauvegarde des informations utilisateur, telles que le nom, le prénom, etc. De plus, les cartes grises seront également enregistrées sous forme de fichiers PNG, JPEG, JPG ou PDF. Les données des utilisateurs seront stockées dans une base de données relationnelle, tandis que seuls les noms des fichiers des cartes grises seront conservés dans cette base. Les fichiers eux-mêmes seront stockés sur le serveur.

Stocker des fichiers provenant d'internet peut paraître dangereux, mais pour réduire les risques, il y'a des bonnes pratiques à mettre en place :

- Autoriser seulement certains types de fichiers
- Authentifier et identifier les utilisateurs qui envoient les fichiers
- Limiter la taille de noms de fichiers et limiter la taille des fichiers en eux même
- Renommer les fichiers avec des noms aléatoires
- Enregistrer les fichiers dans un répertoire non accessible depuis le web
- Utiliser des messages d'erreur simple
 - o Dans le cas où une erreur survient durant la sauvegarder des fichiers, aucune donnée sensible (le chemin du répertoire par exemple) ne doit être divulguée

Source : <https://www.opswat.com/blog/file-upload-protection-best-practices>

4.8. Protection des données

La protection des données des utilisateurs est assurée via les pratiques suivantes :

A. Authentification

La « CA TIC Authentication Gateway » utilisée pour l'authentification au sein de l'application, utilise Microsoft Entra pour l'authentification des utilisateurs, en s'appuyant sur le protocole HTTPS pour sécuriser les connexions.

B. Utilisation du protocole HTTPS

Durant le développement, le protocole HTTP est utilisé pour simplifier les tests. L'application est alors développée et testée localement, avec toutes les données et l'application résidant sur le même ordinateur ou au sein d'un réseau local. Cependant, une fois en production, il est impératif d'utiliser le protocole HTTPS pour garantir la sécurité des données échangées entre les utilisateurs et le serveur.



Figure 13 : HTTP vs HTTPS

4.9. Technologies utilisées

A. Express

Express est un framework web minimaliste pour Node.js qui fournit une série d'outils pour construire des applications web robustes. Il facilite le développement d'API REST et offre des fonctionnalités telles que le routage et les middlewares.

Plus d'informations : <https://expressjs.com>

B. MySQL

MySQL est un système de gestion de bases de données relationnelles open source qui utilise le langage SQL pour gérer les données.

Plus d'informations : <https://dev.mysql.com/doc>

C. Node.js

Node.js est un environnement d'exécution JavaScript côté serveur basé sur le moteur V8 de Google Chrome. Il permet d'exécuter du code JavaScript en dehors du navigateur, et est populaire pour le développement d'applications.

Plus d'informations : <https://nodejs.org>

D. REST / RESTful

REST (Representational State Transfer) est un style architectural utilisé pour concevoir des services web. Les services RESTful sont des API web conformes aux principes REST, utilisant les méthodes HTTP pour interagir avec les ressources.

Plus d'informations : <https://restfulapi.net>

E. Vue

Vue est un framework JavaScript pour créer des interfaces utilisateur. Il se base sur les standards HTML, CSS et JavaScript et offre un modèle de programmation déclaratif et basé sur des composants, facilitant le développement efficace d'interfaces utilisateur de toute complexité.

Plus d'informations : <https://vuejs.org>

5. Conception

5.1. Schéma d'architecture

A. Architecture en tiers

L'architecture multi-tiers, variant d'un (1-tiers) à plusieurs tiers (n-tiers), est une approche de conception logicielle qui sépare une application en différents niveaux fonctionnels.

Dans le cadre du projet, les 3 tiers sont :

1. **Présentation (Navigateur)** : Gère l'interface utilisateur, permettant aux utilisateurs d'interagir avec l'application
2. **Logique (Serveur Web)** : Traite les données, applique les règles métier, et coordonne les actions entre la couche de présentation et celle des données
3. **Données (Base de données)** : Est responsable du stockage, de la récupération, et de la gestion des données

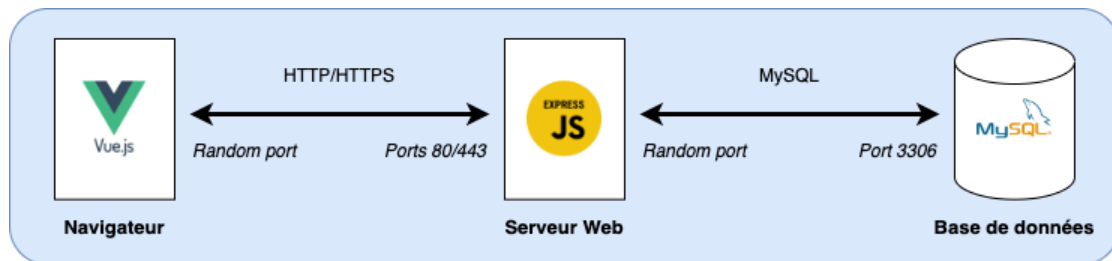


Figure 14 : Architecture 3-tiers

B. Architecture globale

Ci-dessous se trouve un schéma illustrant tous les systèmes utilisés pour l'application. Les relations entre ces systèmes y sont également illustrées.

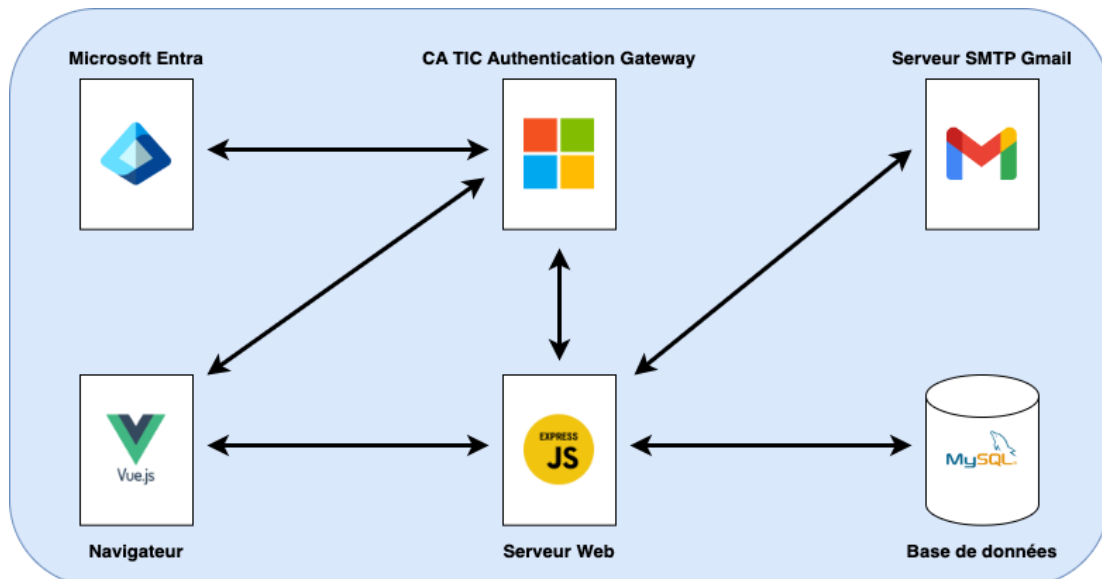


Figure 15 : Architecture globale

5.2. Diagrammes de séquences

A. Authentification

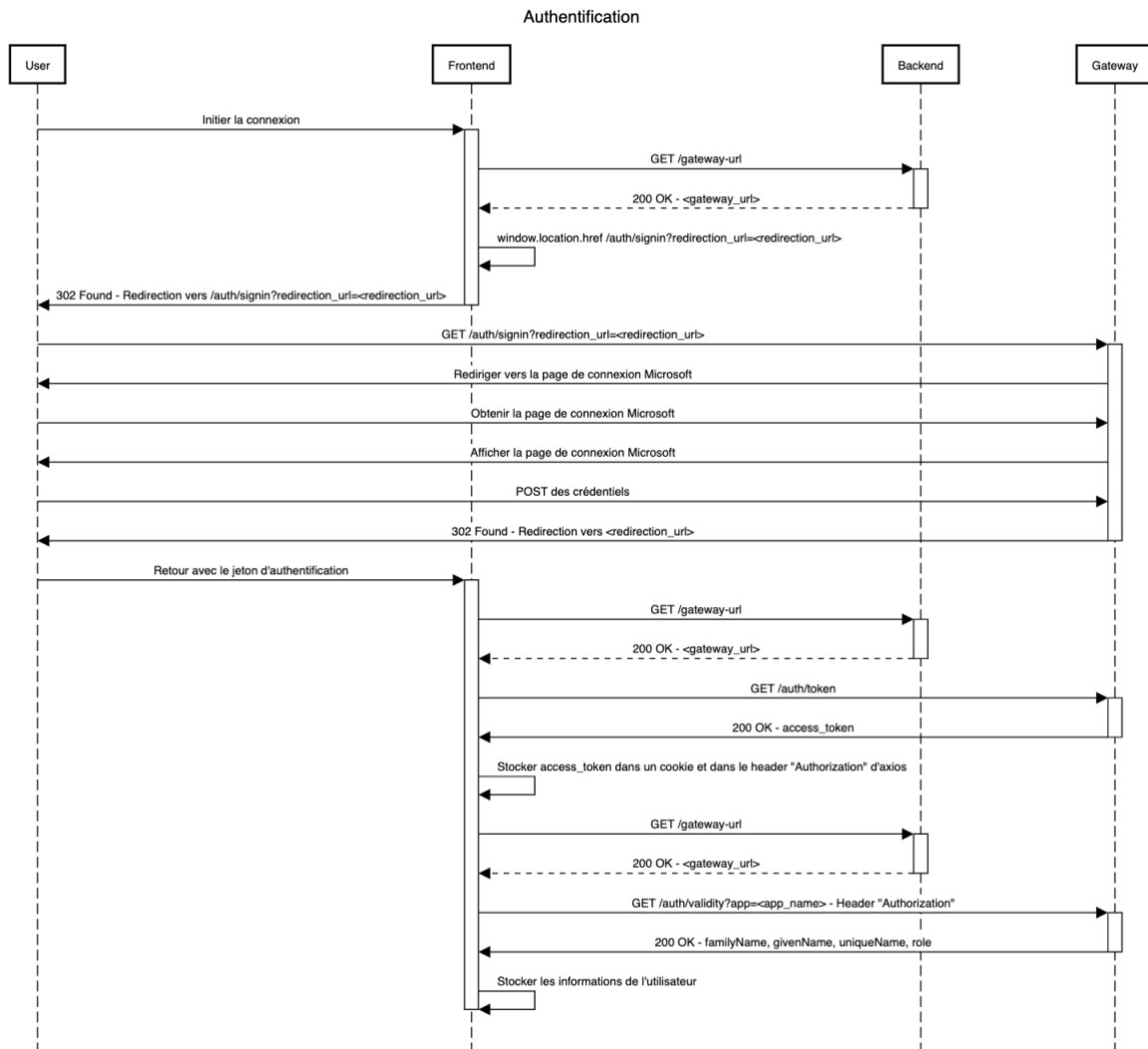


Figure 16 : Diagramme de séquences - Authentification

L'utilisateur initie la connexion dans l'interface Frontend, qui envoie ensuite une requête GET /gateway-url au Backend pour obtenir l'URL de la passerelle d'authentification. Le Backend répond avec l'URL de la passerelle. L'utilisateur envoie une requête à la passerelle et est ensuite redirigé sur la page de connexion Microsoft. L'utilisateur soumet ses informations d'identification via cette page, ce qui génère un jeton d'authentification. Le Frontend envoie une requête GET /auth/token à la Gateway pour obtenir le jeton d'accès, qu'il stocke ensuite dans un cookie et dans le header "Authorization" d'axios. Le Frontend vérifie enfin la validité du jeton et stocke les informations de l'utilisateur.

Événements pouvant perturber le cycle :

- Backend non disponible
- Gateway d'authentification non disponible
- Erreur lors de l'obtention du « access_token »
- Erreur lors de la vérification du « access_token »

B. Envoi d'une demande pour obtenir une vignette

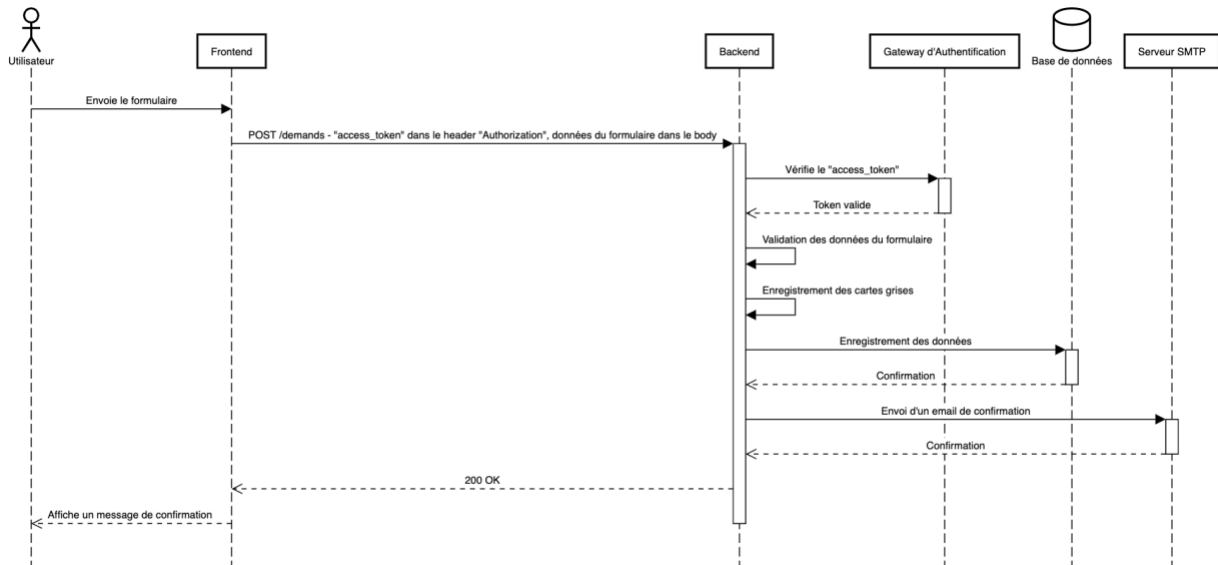


Figure 17 : Diagramme de séquences - Envoi d'une demande pour obtenir une vignette

L'utilisateur remplit le formulaire dans l'interface Frontend, qui envoie ensuite une requête POST /demands au Backend avec le "access_token" et les données du formulaire. Le Backend vérifie le "access_token" auprès de la "Gateway d'Authentification". Après confirmation de la validité du token, le Backend vérifie les données du formulaire, enregistre les cartes grises associées et stocke toutes les informations dans la base de données. Une fois la demande enregistrée, le Backend contacte le serveur SMTP pour envoyer un email de confirmation à l'utilisateur et répond au Frontend avec un code 200 OK. Enfin, un message de confirmation est affiché à l'utilisateur.

Événements pouvant perturber le cycle :

- Backend non disponible
- Gateway d'authentification non disponible
- « Access token » non valide
- Données du formulaire non valides
- Erreur lors de l'enregistrement des cartes grises
- Erreur lors de l'enregistrement des données dans la base de données
- Erreur lors de l'envoi de l'email

C. Affichage de la liste des demandes

Affichage de la liste des demandes

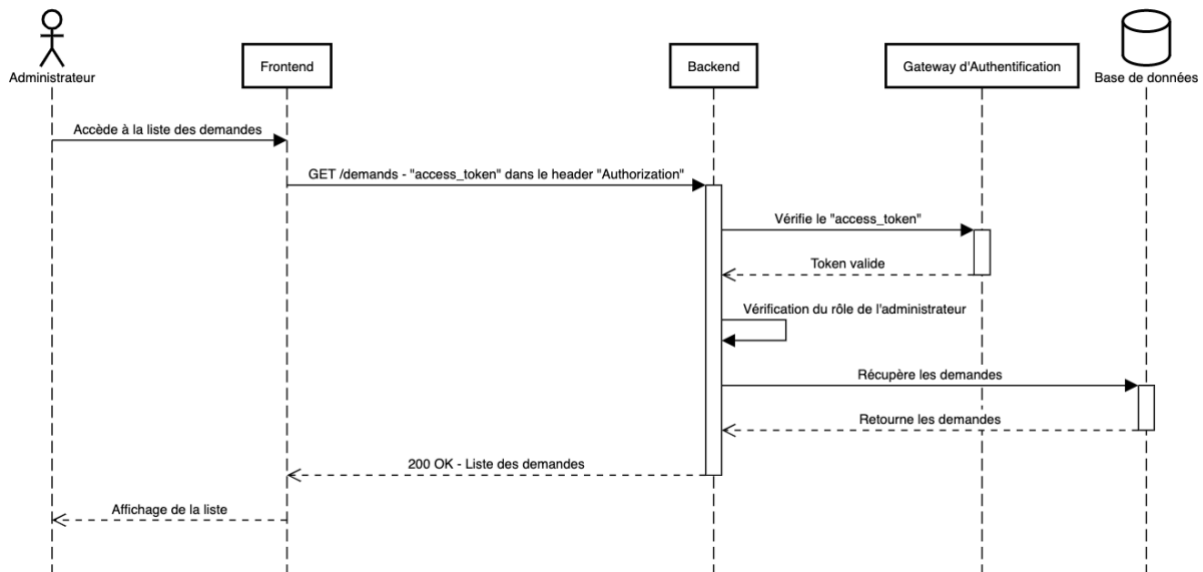


Figure 18 : Diagramme de séquences - Affichage de la liste des demandes

L'administrateur accède à la liste des demandes via l'interface Frontend, qui envoie une requête GET /demands au Backend avec le "access_token". Le Backend vérifie le "access_token" auprès de la "Gateway d'Authentification". Après confirmation de la validité du token, le Backend vérifie le rôle de l'administrateur, récupère les demandes dans la base de données et les renvoie au Frontend avec un code 200 OK. Le Frontend affiche ensuite la liste des demandes à l'administrateur.

Événements pouvant perturber le cycle :

- Backend non disponible
- Gateway d'authentification non disponible
- « Access token » non valide
- Utilisateur avec des rôles non suffisants
- Erreur lors de la récupération des données dans la base de données

D. Exportation de la liste des demandes

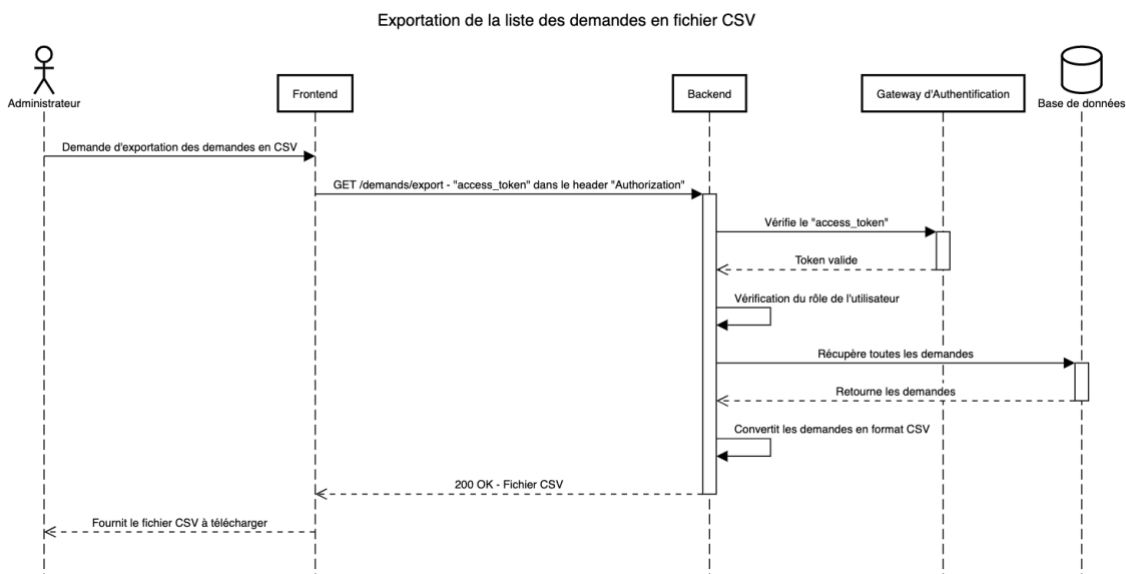


Figure 19 : Diagramme de séquences - Exportation de la liste des demandes

L'administrateur demande l'exportation des demandes en format CSV via l'interface Frontend, qui envoie une requête GET /demands/export au Backend avec le "access_token". Le Backend vérifie le "access_token" auprès de la "Gateway d'Authentification". Après confirmation de la validité du token, le Backend vérifie le rôle de l'administrateur, récupère toutes les demandes dans la base de données et les convertit au format CSV. Le Backend renvoie ensuite le fichier CSV au Frontend avec un code 200 OK, qui le fournit à l'administrateur pour téléchargement.

Événements pouvant perturber le cycle :

- Backend non disponible
- Gateway d'authentification non disponible
- « Access token » non valide
- Utilisateur avec des rôles non suffisants
- Erreur lors de la récupération des données dans la base de données
- Erreur lors de la conversion des données au format CSV

E. Affichage des détails d'une demande

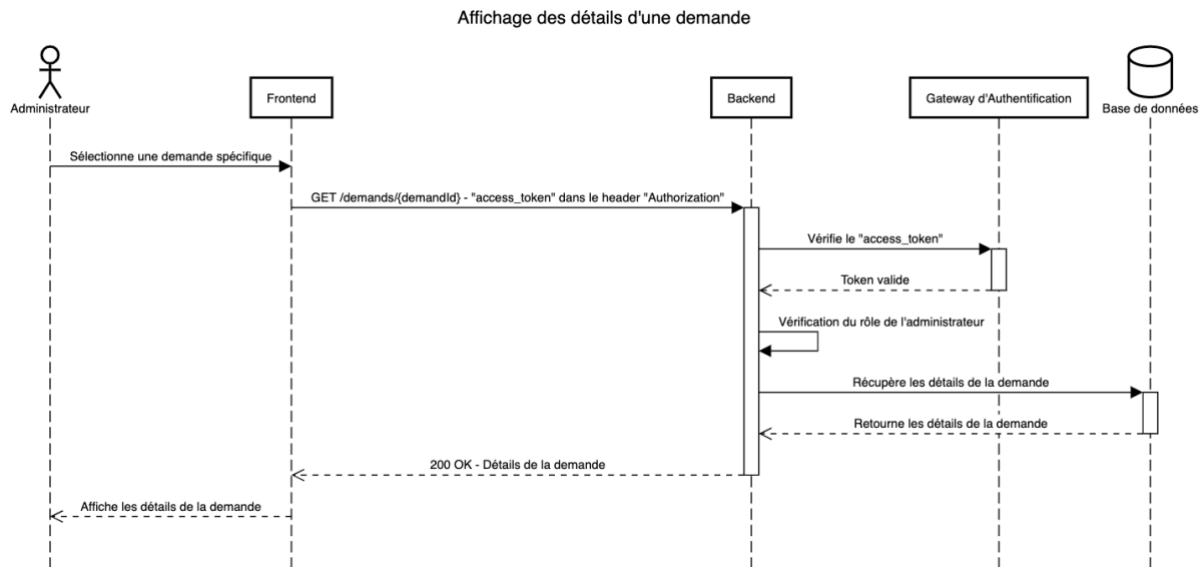


Figure 20 : Diagramme de séquences - Affichage des détails d'une demande

L'administrateur sélectionne une demande spécifique via l'interface Frontend, qui envoie une requête GET /demands/{demandId} au Backend avec le "access_token". Le Backend vérifie le "access_token" auprès de la "Gateway d'Authentification". Après confirmation de la validité du token, le Backend vérifie le rôle de l'administrateur, récupère les détails de la demande dans la base de données, et les renvoie au Frontend avec un code 200 OK. Le Frontend affiche ensuite les détails de la demande à l'administrateur.

Événements pouvant perturber le cycle :

- Backend non disponible
- Gateway d'authentification non disponible
- « Access token » non valide
- Utilisateur avec des rôles non suffisants
- Erreur lors de la récupération des données dans la base de données

F. Téléchargement d'une carte grise



Figure 21 : Diagramme de séquences - Téléchargement d'une carte grise

L'administrateur clique sur un bouton de l'interface Frontend, qui envoie une requête GET `/demands/download/{carRegistrationFilename}` au Backend avec le "access_token". Le Backend vérifie le "access_token" auprès de la "Gateway d'Authentification". Après confirmation de la validité du token, le Backend vérifie le rôle de l'administrateur, récupère la carte grise et la renvoie au Frontend avec un code 200 OK. L'utilisateur reçoit ensuite le fichier.

Événements pouvant perturber le cycle :

- Backend non disponible
- Gateway d'authentification non disponible
- « Access token » non valide
- Utilisateur avec des rôles non suffisants
- Erreur lors de la récupération du fichier

G. Acceptation ou refus d'une demande

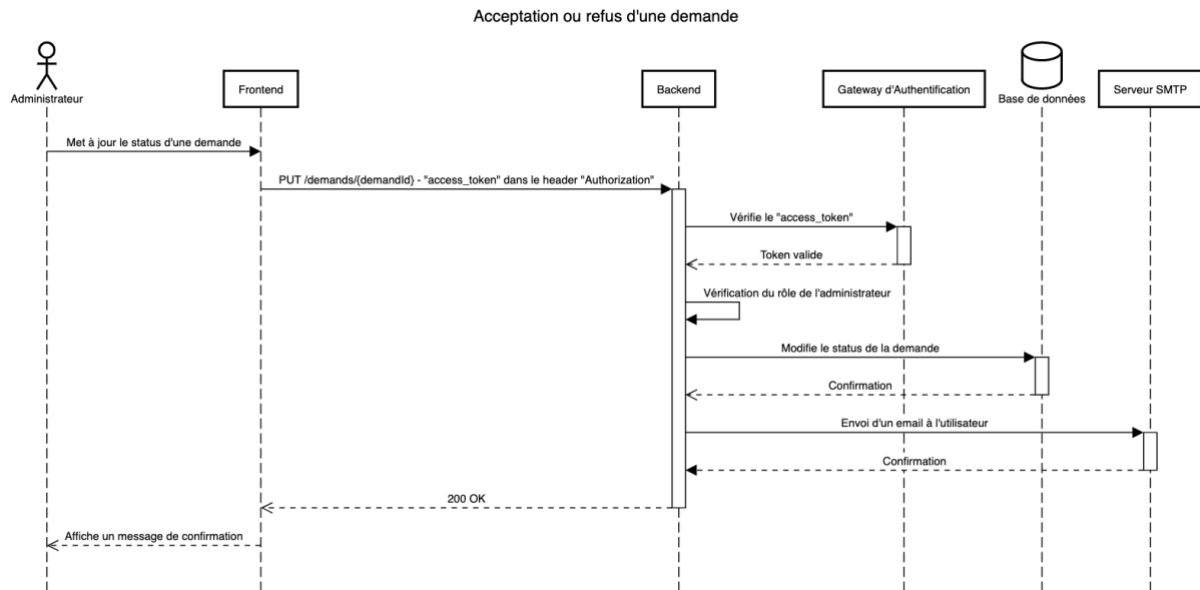


Figure 22 : Diagramme de séquences - Acceptation ou refus d'une demande

L'administrateur met à jour le statut d'une demande via l'interface Frontend, qui envoie une requête PUT /demands/{demandId} au Backend avec le "access_token". Le Backend vérifie le "access_token" auprès de la "Gateway d'Authentification". Après confirmation de la validité du token, le Backend vérifie le rôle de l'administrateur, modifie le statut de la demande dans la base de données, puis envoie une confirmation au Frontend avec un code 200 OK. Le Backend contacte également le serveur SMTP pour envoyer un email à l'utilisateur pour l'informer de l'acceptation ou du refus de sa demande. Enfin, un message de confirmation est affiché à l'utilisateur.

Événements pouvant perturber le cycle :

- Backend non disponible
- Gateway d'authentification non disponible
- « Access token » non valide
- Administrateur avec des rôles non suffisants
- Erreur lors de la mise à jour des données dans la base de données

H. Suppression d'une demande

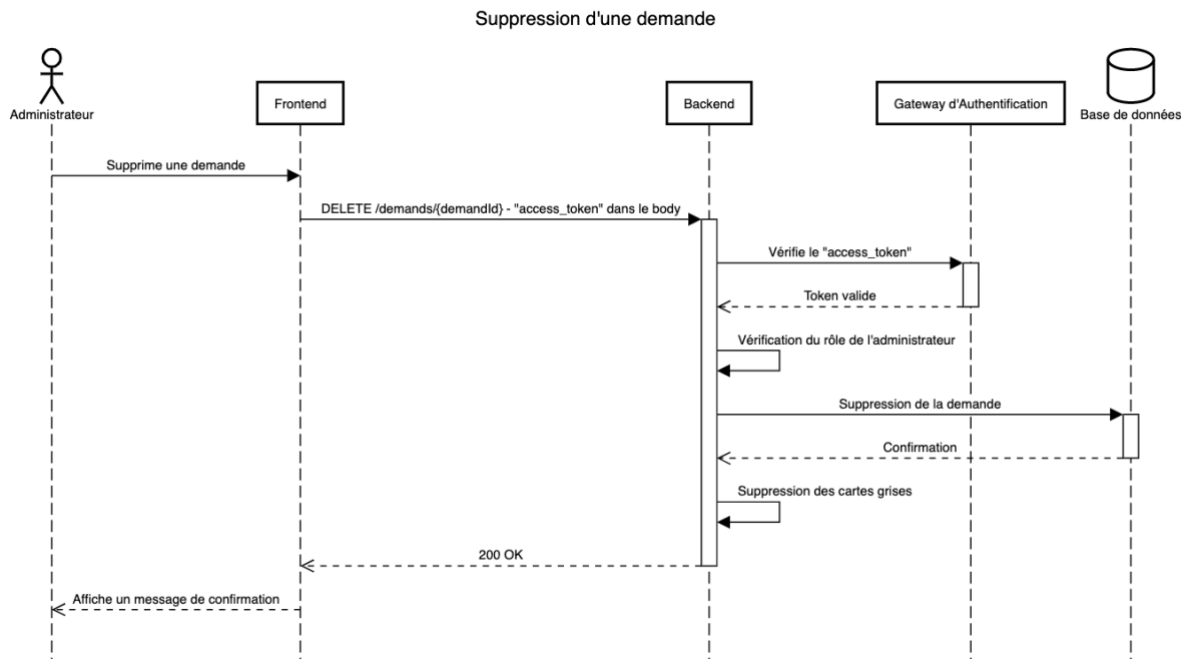


Figure 23 : Diagramme de séquences - Suppression d'une demande

L'administrateur supprime une demande via l'interface Frontend, qui envoie une requête DELETE /demands/{demandId} au Backend avec le "access_token". Le Backend vérifie le "access_token" auprès de la "Gateway d'Authentification". Après confirmation de la validité du token, le Backend vérifie le rôle de l'administrateur, supprime la demande dans la base de données et confirme la suppression au Backend. Ensuite, le Backend procède à la suppression des cartes grises associées. Le Backend renvoie ensuite une confirmation au Frontend avec un code 200 OK. Enfin, un message de confirmation est affiché à l'utilisateur.

Événements pouvant perturber le cycle :

- Backend non disponible
- Gateway d'authentification non disponible
- « Access token » non valide
- Administrateur avec des rôles non suffisants
- Erreur lors de la suppression des cartes grises
- Erreur lors de la suppression des données dans la base de données

5.3. Base de données

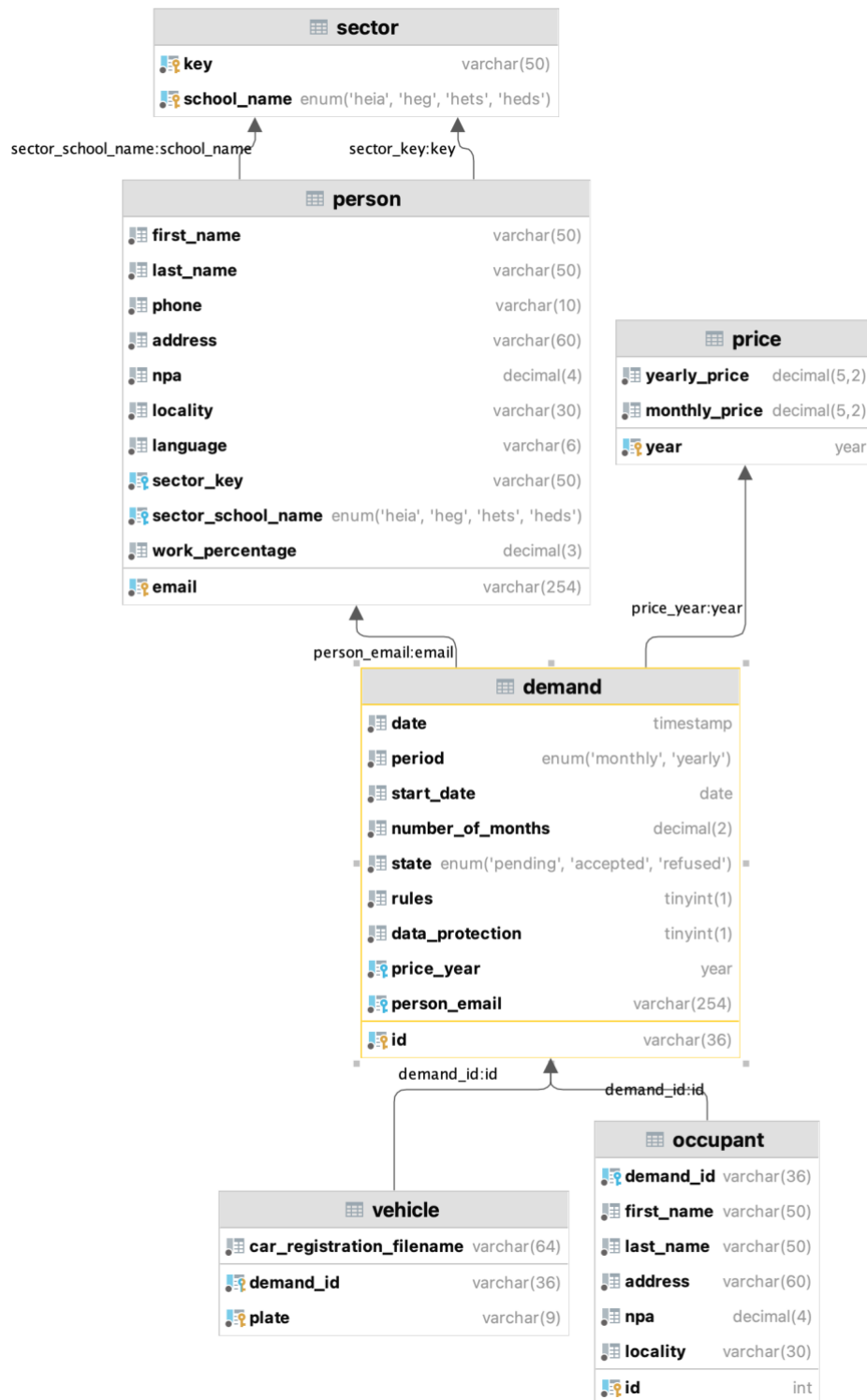


Figure 24 : Base de données

Légende : <https://www.jetbrains.com/help/idea/database-tool-window.html#possible-icon-combinations-for-columns>

A. Tables

La base de données contient six tables, voici une description de chacune d'elles :

Person

Cette table contient les informations des personnes ayant fait des demandes de vignettes. Chaque enregistrement utilise l'email comme identifiant unique. Les informations des personnes sont mises à jour en cas de différences après une nouvelle demande.

Voici une description des différents attributs :

- first_name : Prénom de la personne
- last_name : Nom de famille de la personne
- email : Email de la personne
- phone : Numéro de téléphone de la personne
- address : Adresse de résidence de la personne
- npa : Code postal de la localité de la personne
- locality : Localité de résidence de la personne
- language : Langue préférée de la personne (par exemple, "fr" pour français, "en" pour anglais)
- sector_key : Clé de la filière (identifiant en anglais) permettant de l'identifier
- sector_school_name : Nom de l'école du secteur auquel la personne est associée (HEIA, HEG, HETS, HEDS)
- work_percentage : Pourcentage de travail de la personne

Sector

Cette table contient les informations des différentes filières et écoles associés. Voici une description des différents attributs :

- key : Clé permettant d'identifier la filière (traduction du nom des filières en anglais, espaces remplacés par des « _ ») school_name : Nom de l'école (HEIA, HEG, HETS, HEDS)
- school_name : Nom de l'école dont fait partie la filière (HEIA, HEG, HETS, HEDS)

Il est à noter que chaque combinaison « key » et « school_name » doit être unique.

Demand

Cette table contient les informations des demandes de vignettes.

- id : Identifiant unique pour chaque demande
- date : Date et heure de la demande, par défaut l'horodatage actuel
- period : Période de la demande (mensuelle ou annuelle)
- start_date : Date de début de la demande
- number_of_months : Nombre de mois pour lesquels la demande est faite
- state : État de la demande (en attente, acceptée, refusée)
- rules : Indique si les règles sont acceptées (1 pour vrai, 0 pour faux)
- data_protection : Indique si la protection des données est acceptée (1 pour vrai, 0 pour faux)
- price_year : Année de référence pour le prix
- person_email : Email de la personne ayant fait la demande

Vehicle

Cette table contient les informations des véhicules associés aux demandes.

- demand_id : Identifiant de la demande associée
- plate : Numéro de plaque d'immatriculation du véhicule
- car_registration_filename : Nom du fichier de la carte grise du véhicule (celui-ci étant enregistré sur le serveur). Nomenclature : <demand_id>_<number>.<extension>

Occupant

Cette table contient les informations des occupants associés aux demandes. Étant donné qu'avec les informations actuellement possédées pour les occupants, il n'est pas possible de savoir s'ils sont également membre de la HES-SO//FR, il n'aurait pas été correct dans la table « person ». C'est pour ceci que la table « occupant » a été créée.

Voici une description des différents attributs :

- occupant_id : Identifiant unique pour chaque occupant, auto-incrémenté
- demand_id : Identifiant de la demande associée
- first_name : Prénom de l'occupant
- last_name : Nom de famille de l'occupant
- address : Adresse de l'occupant
- npa : Code postal de l'occupant
- locality : Localité de résidence de l'occupant

Pour éviter les doublons, il est possible de créer des clés uniques pour s'assurer que certaines combinaisons d'attributs, par exemple first_name et last_name dans une voiture, sont uniques. Cependant, cela peut poser un problème car dans ce cas précis, deux personnes avec le même prénom et nom ne pourraient pas être occupants de la même voiture.

Price

Cette table contient les informations des prix des vignettes par année. Voici une description des différents attributs :

- year : Année de référence pour le prix
- yearly_price : Prix annuel de la vignette
- monthly_price : Prix mensuel de la vignette

Il est à noter que l'année est unique pour chaque enregistrement de prix.

B. Types de données

Les types de données pour chaque attribut ont été sélectionnés avec soin, en suivant un ordre de priorité rigoureux pour garantir la pertinence et l'efficacité de la base de données. Voici l'ordre de sélection utilisé :

1. Normes internationales (*RFC*, *ISO*, etc.) : Lorsque des normes reconnues existent, elles ont été prioritaires pour assurer la compatibilité universelle et les meilleures pratiques. Par exemple pour l'email, le errata 1690 de la RFC 3696 a été utilisé : <https://www.rfc-editor.org/errata/errata1690>

2. Normes nationales : Les spécificités nationales, comme les codes postaux, les téléphones ou les plaques de véhicules, ont été prises en compte pour garantir une adéquation avec le contexte national.
3. Adaptation au projet : Les exigences spécifiques du projet, comme les identifiants de filières ou d'écoles, ont été intégrées pour répondre aux besoins opérationnels. Voici l'organigramme de référence pour les écoles et les filières : https://www.hefr.ch/media/xk4qwtl5/hes-so-fr_organigramme_f_2024-01.pdf.
4. Exemples de bases de données existantes : En l'absence de normes spécifiques, des références comme AdventureWorks ont été utilisées pour assurer des pratiques éprouvées et efficaces. Lien vers Adventure Works : <https://dataedo.com/download/AdventureWorks.pdf>

Concernant les identifiants des demandes, le raisonnement est un peu plus spécifique :

- Afin de suivre les bonnes pratiques de sauvegarde de fichiers analysées précédemment, il est nécessaire de les renommer avec un identifiant aléatoire (et ceci avant l'enregistrement de la demande dans la base de données).
- Pour conserver une certaine logique entre la nomenclature des fichiers et l'identifiant des demandes, l'identifiant sera généré à l'aide d'UUIDv4 (<https://www.npmjs.com/package/uuidv4>) avant l'enregistrement de la demande dans la base de données.

C. Cardinalités

Les cardinalités décrivent la nature des relations entre les différentes tables de la base de données, en précisant combien d'enregistrements dans une table peuvent être associés à un enregistrement dans une autre table.

Relation	Type	Justification
person.email demand.person_email	1:N (Un-à-Plusieurs)	Une personne peut faire plusieurs demandes, mais chaque demande est associée à une seule personne.
sector.key person.sector_key	1:N	Un secteur peut avoir plusieurs personnes associées, mais chaque personne est associée à un seul secteur.
sector.school_name person.sector_school_name	1:N	Une école peut avoir plusieurs personnes associées, mais chaque personne est associée à une seule école.
price.year demand.price_year	1:N	Une année de prix peut être utilisée pour plusieurs demandes, mais chaque demande est associée à une seule année.
demand.id vehicle.demand_id	1:N	Une demande peut avoir plusieurs véhicules associés, mais chaque véhicule est associé à une seule demande.

demand.id occupant.demand_id	1:N	Une demande peut avoir plusieurs occupants, mais chaque occupant est associé à une seule demande.
---------------------------------	-----	---

D. Restrictions sur les contraintes d'intégrité référentielle

Les contraintes d'intégrité référentielle sont essentielles pour maintenir la cohérence et la fiabilité des données au sein de la base de données. Elles garantissent que les relations entre les tables restent logiques et que les opérations de mise à jour ou de suppression ne laissent pas de données orphelines ou incohérentes.

Il y'a plusieurs options possibles mais dans le cadre de cette base de données, uniquement 2 seront utilisés :

- **Cascade** : Cette contrainte assure que les actions de suppression ou de mise à jour effectuées sur une table parent se répercutent automatiquement sur les tables enfants reliées. Si une ligne parent est supprimée ou modifiée, toutes les lignes enfants associées sont respectivement supprimées ou mises à jour pour maintenir l'intégrité des données.
- **Restrict** : Cette contrainte bloque la suppression ou la modification d'une ligne dans une table parent si elle est référencée par des lignes dans une table enfant. Cela empêche les actions qui entraîneraient des références orphelines ou des incohérences de données, garantissant que toutes les relations sont intentionnelles et correctement maintenues.

Le tableau ci-dessous décrit les différentes contraintes des relations de la base de données :

Légende : OU=On Update OD=On Delete C=Cascade R=Restrict

Relation	OU	OD	Justification
demand.person_email person.email	C	C	Si l'email d'une personne est mis à jour, les demandes associées sont également mises à jour. Les suppressions sont également en cascade, car l'historique de la personne n'est plus nécessaire si celle-ci doit être supprimée.
person.sector_key sector.key	C	R	Les secteurs doivent être mis à jour dans les enregistrements de personnes. Les suppressions de secteurs encore référencés par des personnes sont bloquées.
person.sector_school_name sector.school_name	C	R	Idem que pour la relation précédente
demand.price_year price.year	R	R	Les années de prix doivent rester invariables si elles sont associées à des demandes.

vehicle.demand_id demand.id	C	C	Si une demande est mise à jour ou supprimée, les véhicules associés sont également mis à jour ou supprimés.
occupant.demand_id demand.id	C	C	Si une demande est mise à jour ou supprimée, les occupants associés sont également mis à jour ou supprimés.

E. Normalisation

La normalisation dans le contexte des bases de données relationnelles est un processus de conception qui vise à minimiser la redondance et la dépendance des données. Pour normaliser la base de données, il y'a plusieurs formes à respecter :

- **Première Forme Normale (1NF)** : Une relation (ayant par définition un identifiant) est en première forme normale si tous les attributs possèdent tous une valeur sémantiquement atomique.
- **Deuxième Forme Normale (2NF)** : Les colonnes non-clés, doivent dépendre entièrement de la clé primaire.
- **Troisième Forme Normale (3NF)** : Les colonnes non-clés, ne doivent pas dépendre d'autres colonnes non-clés.

Il est à noter que chaque forme normale doit respecter la précédente, donc la 3NF respecte la 2NF qui elle-même respecte la 1NF.

Source : <https://api-ti.pages.forge.hefr.ch/a12/db/#normalisation>

5.4. Mockups

Un mockup est un design d'une application, qui illustre à quoi pourrait ressembler le résultat final. Bien que le design final de l'application puisse être sujet à des modifications, la structure de base reste généralement inchangée.

 **EPARKING** Administration Déconnexion

Demande d'une d'une vignette de stationnement

Informations personnelles

Informations du véhicule

[Ajouter un véhicule](#)

Autre(s) occupant(s) du véhicule

[Ajouter un occupant](#) [Supprimer un occupant](#)

Période

☒ Mensuelle (35.-/mois) ☐ Annuelle (420.-/an)

☐ Je déclare avoir pris connaissance des [règles concernant les vignettes et l'utilisation des parkings du plateau de Pérolles](#) et en accepte le contenu.
☒ Je déclare avoir pris connaissance des [conditions de protections des données](#) et en accepte le contenu.

Envoyer

Besoin d'aide ? Contactez nous par email à l'adresse eparking@hefr.ch ou par téléphone au 026 429 00 00

Figure 25 : Mockup - Formulaire - Format ordinateur



[Formulaire](#) [Déconnexion](#)

Liste des demandes

[AFFICHER LES DEMANDES TRAITÉES](#)

[EXPORTER](#)

Nom	Prénom	Ecole	NPA et Localité	Nombre d'occupants	Détails
Rochat	Daniel	HEIA	1563 Dompierre	1	🔍
Gerber	Peter	HETS	1630 Bulle	0	🔍
Müller	Thomas	HEG	1634 La Roche	2	🔍
Schmid	Hans	HEDS	1700 Fribourg	1	🔍
Fournier	Christian	HEDS	1752 Villars-sur-Glâne	3	🔍
Favre	Martin	HETS	1741 Cottens	4	🔍
Kolly	Michael	HEIA	1544 Gletterens	0	🔍
Wüthrich	Andreas	HEIA	1637 Charmey	1	🔍
Schneider	Markus	HEG	1470 Estavayer-le-Lac	0	🔍
Aebischer	David	HEIA	1656 Jaun	2	🔍

Rows per page: 10 ▼ 1-10 of 20 [◀](#) [▶](#)

Besoin d'aide ? Contactez nous par email à l'adresse eparking@hefr.ch ou par téléphone au 026 429 00 00

Figure 26 : Mockup - Liste des demandes - Format ordinateur

 **EPARKING**

Formulaire Déconnexion

Détails de la demande

 470702f2-ab2a-45e8-a69c-313aba2105c

 2023-12-31 23:59:59

 En attente

Informations personnelles

Daniel	Rochat	
daniel.rochat@hefr.ch	0790000000	
Route du Lac 1	1563	Dompiere
HEIA	HEIA	100

Informations du véhicule

FR123456	470702f2-ab2a-45e8-a69c-313aba2105c2_1.png 
----------	--

Autre(s) occupant(s) du véhicule

Jeanne	Rochat	
Route du Lac 1	1563	Dompiere

Période

☒ Mensuelle (35.-/mois) ☐ Annuelle (420.-/an)

Juillet 2024

Février 2025

☒ Je déclare avoir pris connaissance des règles concernant les vignettes et l'utilisation des parkings du plateau de Pérolles et en accepte le contenu.

☒ Je déclare avoir pris connaissance des conditions de protections des données et en accepte le contenu.

Accepter Refuser 

Besoin d'aide ? Contactez nous par email à l'adresse eparking@hefr.ch ou par téléphone au 026 429 00 00

Figure 27 : Mockup - Détails d'une demande - Format ordinateur


EPARKING
Administration Déconnexion

Demande d'une vignette de stationnement

Informations personnelles

Prénom

Nom

Email

Téléphone

Adresse

NPA

Localité

Ecole

Filière / Institut


EPARKING
Formulaire Déconnexion

Liste des demandes

EXPORTER

VOIR LES DEMANDES TRAITÉES

Rechercher

Nom	Prénom	Ecole
Rochat	Daniel	HEIA
Gerber	Peter	HETS
Müller	Thomas	HEG
Schmid	Hans	HEDS
Fournier	Christian	HEDS
Favre	Martin	HETS
Kolly	Michael	HEIA
Wüthrich	Andreas	HEIA
Schneider	Markus	HEG
Aebischer	David	HEIA


EPARKING
Formulaire Déconnexion

Détails de la demande

 470702f2-ab2a-45e8-a69c-313aba2105c

 2023-12-31 23:59:59

 En attente

Informations personnelles

Daniel

Rochat

daniel.rochat@hefr.ch

0790000000

Route du Lac 1

1563

Dompiere

Figure 28 : Mockups - Format mobile

5.5. Stratégie de tests

Une stratégie de test a été élaborée pour identifier les éventuelles erreurs ou dysfonctionnements de l'application. Pour ce faire, plusieurs méthodes ont été mises en œuvre afin de garantir qu'aucune erreur ne subsiste dans l'application :

- Tests de compatibilité sur plusieurs navigateurs : Edge, Firefox, Chrome, Opera, Safari.
- Tests de compatibilité sur plusieurs systèmes d'exploitation : MacOS, Windows
- Beta test réalisé par des tierces personnes.
- Test de chaque fonctionnalité séparément.
- Test de l'ensemble de l'application.

Ces tests seront réalisés de manière continue tout au long du développement du projet afin d'assurer le bon fonctionnement de l'application.

6. Réalisation

6.1. Structure du projet

Racine	
Fichiers/Répertoires	Description
.gitignore	Liste les fichiers à exclure du contrôle de version.
package.json, package-lock.json	Fichiers de configuration pour npm, listant les dépendances de développement et leurs versions respectives.
.eslintrc.json, .prettierrc.json	Fichiers de configuration pour ESLint et Prettier, définissant les règles de style et de formatage du code.
scripts/	Répertoire contenant des scripts, notamment pour la création et la gestion de la base de données.

Backend	
Fichiers/Répertoires	Description
package.json, package-lock.json	Fichiers de configuration pour npm, listant les dépendances du backend et leurs versions respectives.
.env.development, .env.poduction	Contient les variables d'environnement spécifiques aux environnements de développement et de production.
index.js	Fichier principal qui initialise et configure l'application.
mailTexts/	Contient les textes des emails dans les différentes langues disponibles.
middlewares/	Contient les middlewares (authentification, validation du formulaire)
routes/	Contient les différentes routes de l'application (par exemple, demands.js pour la route /demands).
services/	Contient les services pour des fonctionnalités telles que la gestion de la base de données, des fichiers, et des emails.
utils/	Contient les utilitaires, comme le logger pour la journalisation.

Frontend	
Fichiers/Répertoires	Description
package.json, package-lock.json	Fichiers de configuration pour npm, spécifiant les dépendances du frontend et leurs versions respectives.
env.development, .env.poduction	Contient les variables d'environnement spécifiques aux environnements de développement et de production.
jsconfig.json et vite.config.js	Fichiers de configuration pour le compilateur JavaScript et Vite, respectivement.
src/App.vue	Composant principal de l'application Vue.js.
src/main.js	Fichier d'entrée principal pour initialiser l'application Vue.js.
src/assets/	Contient les ressources statiques comme les images et les styles CSS.
src/components/	Contient les différents composants Vue.js utilisés dans l'application.
src/i18n/	Contient les fichiers de traduction pour l'internationalisation.
src/router/	Contient la configuration du routeur Vue.js pour la navigation entre les pages.
src/services/	Contient les différents services pour des fonctionnalités comme l'authentification et les appels backend.
src/stores/	Contient les différents stores pour la gestion de l'état global de l'application, comme l'état de l'utilisateur.

6.2. Linter et code formatter

Pour garantir un code clair, lisible et prêt pour la production, ESLint et Prettier sont installés à la racine du projet comme dépendances de développement. ESLint vérifie les erreurs de syntaxe tandis que Prettier s'occupe du style du code. Pour assurer une intégration harmonieuse entre les deux, le plugin "eslint-config-prettier" est également installé, évitant ainsi toute contradiction entre les règles des deux outils.

La plupart des règles sont configurées automatiquement, mais deux ont été ajustées manuellement afin d'avoir un code uniforme :

- Interdire l'utilisation des logs dans la console afin que le code soit prêt pour la production.
- Imposer l'utilisation des guillemets simples pour uniformiser le style du code.

ESLint : <https://eslint.org/>

Prettier : <https://prettier.io/>

6.3. Layout et design

A. Vuetify

Vuetify est une bibliothèque UI conçue pour Vue.js. Elle comprend une vaste gamme de composants préconstruits qui permettent de faciliter la création d'interfaces utilisateurs ergonomiques. Cette bibliothèque possède 4 grands avantages :

- **Basé sur le material design** : Assure une uniformité visuelle dans l'ensemble des pages. Plus d'informations : https://fr.wikipedia.org/wiki/Material_Design.
- **Support par les navigateurs** : Compatible avec une large gamme de navigateurs, y compris tous les navigateurs basés sur Chromium (Chrome, Opera, Edge), ainsi que Firefox et Safari.
- **CSS Reset** : Permet de normaliser les styles par défaut des éléments HTML, assurant une base de style uniforme et réduisant les incohérences entre les navigateurs.
- **Responsive Design** : Tous les composants sont responsives et certains possèdent des modes spécifiques pour les mobiles, garantissant une adaptabilité sur tous les appareils.

Source : <https://vuetifyjs.com/>

B. Internationalisation

L'internationalisation (abrégée i18n), est le processus de concevoir une application pour qu'elle puisse être adaptée dans différentes langues. Avec Vue.js, il est possible d'utiliser un plugin nommé « Vue I18n ». Source : <https://vue-i18n.intlify.dev/>.

Dans le cadre de ce projet, les traductions sont stockées dans un fichier « fr.json » qui se trouve dans le répertoire « i18n » du Frontend. Les traductions sont ensuite accessibles dans le template avec la variable « \$t » suivi du pointeur JSON (par exemple \$t('demandDetails') donne « Détails de la demande »). Les traductions peuvent également être utilisées dans les scripts, mais cette fois ci en utilisant « i18n.t » à la place de « \$t ».

6.4. Gestion des configurations

A. Variables d'environnement

Afin d'éviter d'avoir à parcourir tous les fichiers du code pour modifier des configurations, toutes les variables spécifiques à l'environnement dans lequel fonctionne l'application sont centralisées et définies dans des fichiers « .env », sous forme de paires « clé-valeur ». Par exemple pour stocker un mot de passe de la base de données, ceci serait utilisé :

```
DB_PASSWORD=mysecurepassword
```

Avec Vue.js, les variables sont directement utilisables avec l'objet « import.meta.env » tant que le nom de la variable commence par « VITE ».

```
console.log(import.meta.env.MY_VARIABLE) // undefined
console.log(import.meta.env.VITE_MY_VARIABLE) // «my_value»
```

Source : <https://vitejs.dev/guide/env-and-mode>

En revanche pour Express, un module nommé dotenv doit être installé. Une fois installé, les variables d'environnement peuvent être utilisées avec l'objet « process.env ».

```
console.log(process.env.MY_VARIABLE) // «my_value»
```

Source : <https://www.npmjs.com/package/dotenv>

B. Gestion des modes

Les fichiers « .env » peuvent être configurés pour différent mode. En effet, il n'est pas toujours désirable d'utiliser les mêmes configurations pour le développement et pour la production. Pour ce faire, il faut spécifier le mode dans le nom du fichier. Ainsi, « .env.development » pourra être utilisé pour le développement, et « .env.production » pour la production.

Ensuite, la sélection du bon fichier se fait lors de l'exécution :

- **Vue.js** : Sélection selon la commande utilisée. La commande « Vite » utilisera le fichier de développement, tandis que la commande « Vite build » utilisera le fichier de production.
- **Express** : Doit être précisé dans le commande d'exécution : « NODE_ENV=<mode> npm start » utilisera le fichier « .env.<mode> », et inversement pour le mode production.

6.5. Authentification

A. Gestion des utilisateurs et des droits

En utilisant la « CA TIC Authentication Gateway », tous les utilisateurs faisant partie du tenant Microsoft de la HES-SO se verront accorder l'accès à l'application. Néanmoins, sans configuration manuelle, aucun droit n'est accordé aux utilisateurs.

Les droits accordés aux utilisateurs sur l'application doivent donc se faire au travers de rôle, attribués aux utilisateurs directement dans une base de données SQLite sur le serveur de la gateway. Voici ce à quoi elle ressemble :

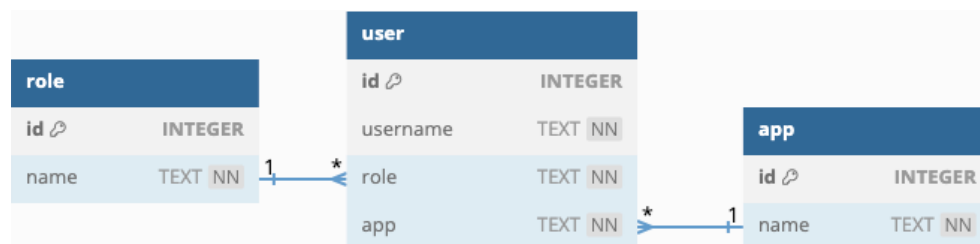


Figure 29 : Base de données - CA TIC Authentication Gateway

Voici la procédure à suivre afin de donner des rôles :

- Ajouter l'application dans la base de données

```
INSERT INTO app (name) VALUES ('eparking')
```

- Ajouter les rôles souhaités

```
INSERT INTO role (name) VALUES ('admin')
```

- Ajouter un utilisateur, spécifier le rôle ainsi que l'application.
 - o Le username est l'adresse email « HES-SO » de l'utilisateur en question.

```
INSERT INTO user (username,role,app) VALUES ('user@hes-so.ch','admin',  
, 'eparking')
```

B. Frontend

L'authentification dans le frontend de l'application permet de garantir que les utilisateurs autorisés puissent avoir accès aux différentes fonctionnalités de l'application. En effet, le processus permet non seulement d'avoir accès aux différentes interfaces, mais également d'obtenir un « access_token » permettant d'accéder aux ressources protégées du backend.

Vue Router (frontend/src/router/index.js) :

Lorsqu'un utilisateur veut accéder à l'application, le router permet d'interpréter le chemin saisi et de diriger l'utilisateur vers le bon composant. Les routes sont définies par 3 attributs :

```
{  
  path: '/admin', // Chemin  
  name: 'DemandsList', // Nom de la route  
  component: DemandsList, // Composant Vue  
}
```

Pour protéger ces routes, il est possible d'utiliser des « Navigation Guards ». Comme leur nom l'indique, ces « Navigation Guards » permettent d'autoriser ou de refuser des navigations selon des critères définis. Dans le cadre d'eParking, il y'a 2 « Guards » à mettre en place :

- **requiresAuth** : Qui autorise seulement les utilisateurs authentifiés
- **requiresAdmin** : Qui autorise seulement les utilisateurs authentifiés possédant le rôle « admin »

Il est donc possible de protéger les routes nécessitant un rôle « admin » comme ceci :

```
{  
  path: '/admin',  
  name: 'DemandsList',  
  component: DemandsList,  
  meta: {  
    requiresAuth: true,  
    requiresAdmin: true,  
  },  
}
```

Une fois les routes et les « Guards » définis, il est possible d'utiliser « router.beforeEach » pour que des actions soient accomplies avant chaque navigation. Pour le « Guard » requiresAuth par exemple :

```
if (to.meta.requiresAuth) {  
  const isAuthenticated = await authenticateUser();  
  if (!isAuthenticated) {  
    return;  
  }  
}
```

Dans ce cas précis, la constante « isAuthenticated » fait appel à la fonction « authenticateUser » du service « authentication.js ». Il la fonction retourne « true », l'utilisateur pourra naviguer, dans le cas contraire, le service d'authentification s'occupera de le rediriger vers la page d'authentification Microsoft.

Source : <https://router.vuejs.org/guide/advanced/navigation-guards.html>

Service d'authentification (frontend/src/services/authentication.js) :

Ce service gère l'authentification un utilisateur. Dans un premier temps, voici comment fonctionne la fonction « authenticateUser » :

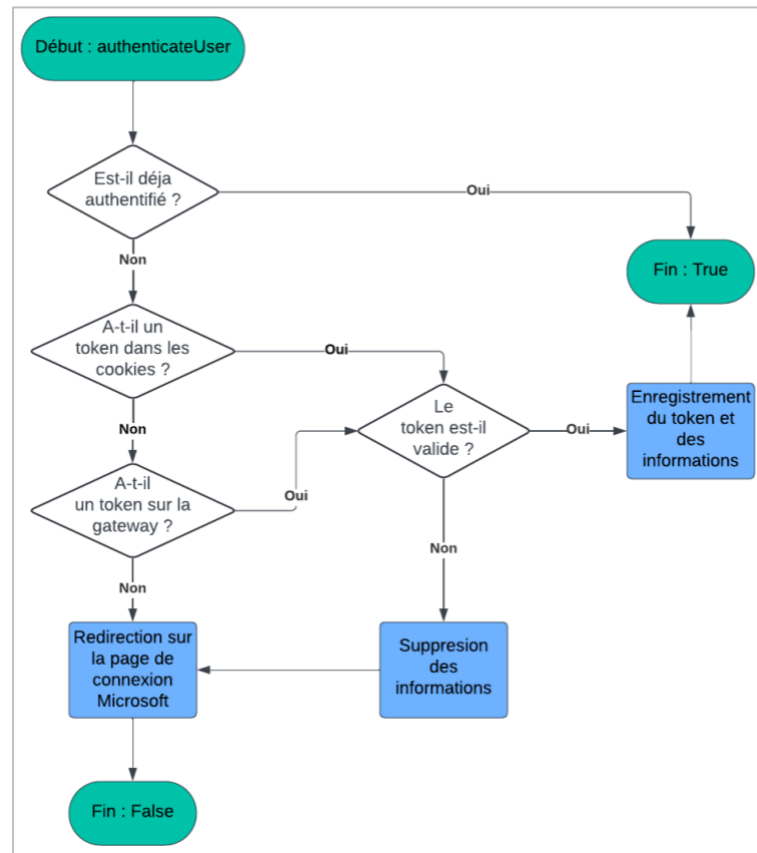


Figure 30 : Diagramme de flux - authenticateUser

Dans un premier temps, une vérification du « userStore » est effectuée pour savoir si l'utilisateur est déjà authentifié. Si c'est le cas, la fonction retourne « true ». Dans un second temps, le contenu du cookie « access_token » est récupéré. Si le contenu est vide, une requête vers la gateway pour récupérer un « access_token » est effectuée. Si après ces 2 opérations un « access_token » est obtenu, celui-ci sera validé via gateway, les informations du token seront sauvegardées dans le « userStore » et la fonction retournera « true ». Dans le cas où aucun « access_token » n'est trouvé ou que la vérification échoue, le « userStore » et le cookie seront vidés pour s'assurer qu'il ne reste aucune information erronée, l'utilisateur sera redirigé vers la page de connexion Microsoft, et la fonction retournera « false ».

userStore (frontend/src/stores/user.js)

Le store utilisateur est géré avec Pinia et maintient l'état de l'utilisateur. Il contient les informations de l'utilisateur, leur rôle, et fournit des actions et getters pour manipuler et accéder à cet état.

Les actions sont les suivantes :

- setUser : Définit les informations de l'utilisateur
- clearUser : Efface les informations de l'utilisateur

Les getters sont les suivants :

- isAuthenticated : Vérifie si l'utilisateur est authentifié
- isAdmin : Vérifie si l'utilisateur possède un rôle d'administrateur
- getUser : Récupère les informations de l'utilisateur

Source : <https://pinia.vuejs.org/>

Service backend (frontend/src/services/backend.js)

Une fois que l'utilisateur a été autorisé à naviguer par les « Guards » du router, il aura accès aux différentes pages du frontend. Pour la fonctionnalité « Affichage de la liste des demandes » par exemple, des données doivent être récupérées depuis le backend, puisque c'est le backend qui interagit avec la base de données. C'est la librairie Axios qui se charge d'envoyer ces requêtes.

Le backend effectue une nouvelle vérification du « access_token » qui lui est transmis et de ce fait, deux conséquences en découlent :

- Les requêtes vers le backend doivent avoir le « access_token » dans le header « Authorization »
- Si une altération du token a eu lieu, il est possible que le backend renvoie un code d'erreur 401 ou 403.

Pour traiter ces cas, Axios permet la mise en place d'intercepteurs. Ceux-ci se placent entre l'utilisateur et l'appel http et permettent d'intercepter les requêtes avant qu'elles ne soient envoyées, ou les réponses avant qu'elles n'atteignent l'utilisateur.

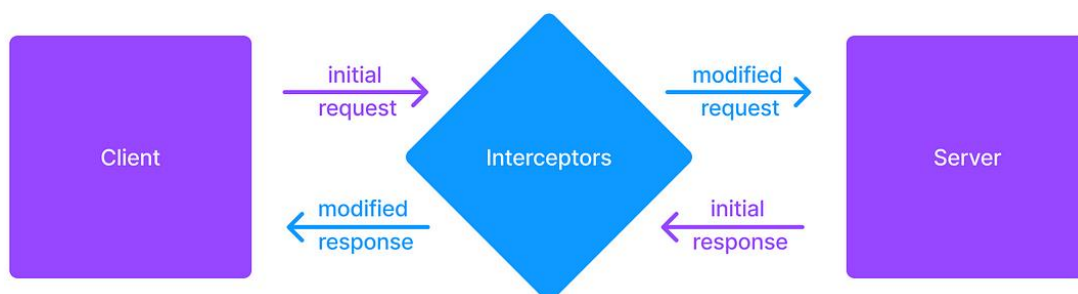


Figure 31 : Intercepteurs Axios

Deux intercepteurs doivent donc être configurés :

- **Intercepteur de requête:** Ajoute le « access_token » dans le header « Authorization » des requêtes avant qu'elles ne soient envoyées au backend.
- **Intercepteur de réponse:** Gère les erreurs retournées par le backend. Si une erreur d'authentification (HTTP Status Code 401 ou 403) est détectée, l'utilisateur est redirigé vers une page d'erreur appropriée et les informations de l'utilisateur sont effacées. Quand celui-ci tentera d'accéder à une nouvelle page, il sera redirigé vers la page de connexion par le service d'authentification.

Source : <https://axios-http.com/docs/interceptors>

C. Backend

Le backend, doit également vérifier les « access_token » car comme pour le frontend, les routes ont différents niveaux d'accès.

Pour ce faire, Express permet l'utilisation de « middlewares ». Ces middlewares agissent comme des intercepteurs de requêtes, entre les routes et l'utilisation des services. Ils permettent notamment la vérification des informations de la requête (comme le « access_token »).

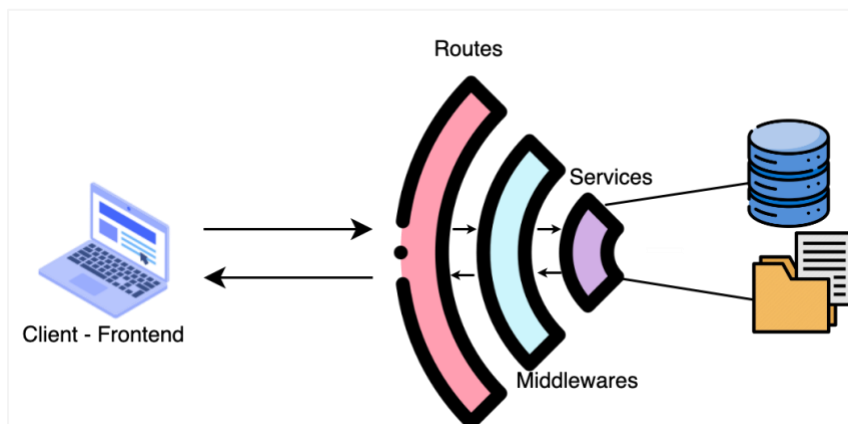


Figure 32 : Middlewares

Il y'a donc 2 middlewares à créer :

- **isAuthenticated** : Qui autorise seulement les utilisateurs authentifiés
- **isAdmin** : Qui autorise seulement les utilisateurs authentifiés possédant le rôle « admin »

Par défaut, les routes Express sont définies comme suit :

- Une personne se rendant donc sur <backend_url>/public recevra « Ceci est une route publique ».

```
app.get('/public', (req, res) => {  
  res.send('Ceci est une route publique.');
```

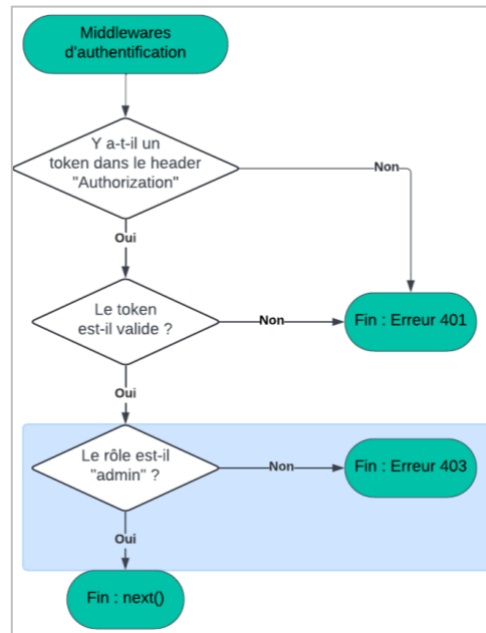
```
});
```

Avec les middlewares, elles doivent être définies comme ceci :

```
// Route protégée accessible uniquement aux administrateurs  
app.get('/admin', isAuthenticated, isAdmin, (req, res) => {  
  res.send('Ceci est une route pour les administrateurs');
```

```
});
```

A l'intérieur des middlewares, se fait en suivant ce procédé :



Explications :

La première étape consiste à récupérer le header « Authorization » contenant le « access_token », puis à le vérifier avec la gateway d'authentification. Si le header est vide ou que la validation échoue, cela signifie que l'utilisateur n'est pas correctement authentifié, ce qui donne une erreur 401 (Non Authorized).

La partie bleue est utilisée seulement lorsque le middleware « isAdmin » est utilisé. La réponse de la validation du « access_token » étant sous la forme ci-dessous, il est possible de vérifier le rôle de l'utilisateur sans faire de nouvelle requête. Si le rôle n'est pas « admin », une erreur 403 lui sera renvoyée (Forbidden).

Source : <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>

6.6. Validation des données du formulaire

Les données du formulaire doivent être validées 2 fois. La première validation s'effectue du côté client, permet d'aider l'utilisateur à identifier ses erreurs et à les corriger. Cependant, rien n'empêche l'utilisateur de passer outre les règles en modifiant le frontend depuis son navigateur ou en passant par un autre client http.

A. Frontend

Pour la validation client, Vuetify propose un système de validation basé sur des règles définies dans un fichier spécifique, appliquées via l'attribut « rules » des champs de texte des formulaires. Pour centraliser ces règles, et leur permettre d'être éventuellement réutilisées dans plusieurs composants, créer un service et importer les règles définies est la meilleure option.

Par exemple dans le service « formValidator », la règle pour la validation du prénom est la suivante :

```
firstNameRules: [  
  (v) => !!v || t('form.rules.firstName.required'),  
  (v) =>  
    (v && v.length <= 50) || t('form.rules.firstName.maxLength', { max:  
50 } ),  
],
```

Une fois importée, la règle peut être utilisée directement sur le champ de texte.

```
<v-text-field  
  v-model="form.firstName"  
  :label="$t('form.personalInformationInputs.firstName')"  
  :rules="rules.firstNameRules"  
></v-text-field>
```

Si l'utilisateur tente de remplir le champ en ne respectant pas les conditions requises pour le prénom, la bordure du champ sera rouge et l'erreur lui sera indiquée. De plus, le bouton d'envoi du formulaire sera désactivé.



Figure 33 : Validation des données du formulaire - Frontend

B. Backend

Pour la validation du côté backend, un middleware « formValidator » est utilisé. Les règles identiques à celle du frontend sont tout d'abord définies :

```
const validateFirstName = (firstName) => {  
  if (!firstName) return errorMsg('First name is required');  
  if (firstName.length > 50)  
    return errorMsg('First name must be less than 50 characters');  
  return null;  
};
```

Puis elles sont ensuite appliquées aux données :

```
const errors = [  
  validateFirstName(form.firstName),  
  // Autres vérifications....  
]
```

Une fois toutes les validations effectuées, le tableau « Errors » est soit vide dans le cas où aucune erreur n'a eu lieu, soit rempli avec les messages d'erreur dans le cas où des données étaient incorrectes.

S'il y'a eu des erreurs durant la validation, le middleware bloque l'accès à la route en renvoyant un code HTTP 400 (Requête invalide), avec la liste des erreurs.

Source : <https://developer.mozilla.org/fr/docs/Web/HTTP/Status/400>

6.7. Gestion des fichiers

En utilisant Express, il n'est pas nécessaire d'installer une librairie supplémentaire pour la gestion des fichiers, car le module natif « fs » (file system) de Node.js est déjà disponible. Toutes les fonctions de gestion des fichiers sont centralisées dans le service « fileManager ».

Pour suivre les principes de sécurité établis lors de l'analyse, tous les fichiers sont renommés avec un identifiant unique de la demande à laquelle ils sont liés, généré aléatoirement avec uuidv4. Tous les fichiers doivent être stockés dans un répertoire sécurisé, non accessible depuis le web.

De plus, toutes les interactions visant à récupérer des fichiers (par exemple, pour le téléchargement des cartes grises) sont réservées aux administrateurs, en utilisant le middleware isAdmin pour assurer la sécurité des accès.

Il est également notable que lorsqu'une demande est supprimée, les cartes grises associées doivent également l'être pour éviter de conserver des fichiers orphelins.

Source FS : <https://nodejs.org/api/fs.html#file-system>

Source UUIDv4 : <https://www.npmjs.com/package/uuid>

6.8. Base de données

A. Librairie

Il existe plusieurs bibliothèques pour se connecter aux bases de données avec Express. Parmi elles, mysql et mysql2 sont les plus populaires. Après avoir comparé ces deux options, il a été conclu que mysql2 est la meilleure solution. En effet, bien que les deux bibliothèques offrent les fonctionnalités nécessaires pour ce projet, mysql n'a pas été mis à jour depuis plus de 4 ans et a presque trois fois moins de téléchargements que mysql2. Cela entraîne une communauté moins active et une documentation moins abondante pour mysql.

<https://npm trends.com/mysql-vs-mysql2>

B. Utilisateur

Pour assurer une sécurité optimale, il est essentiel de créer un utilisateur dédié spécifiquement pour la base de données du projet, en respectant le principe du moindre privilège. Voici les droits qui doivent lui être accordés sur la base de données :

- **SELECT** : Afin de pouvoir accéder à des données de la base
 - Récupérer des détails sur les demandes, etc...
- **INSERT** : Afin de pouvoir enregistrer des données dans la base
 - Insérer un nouvel utilisateur, une nouvelle demande, etc...
- **UPDATE** : Afin de pouvoir mettre à jour des données
 - Mettre à jour un utilisateur (s'il refait une demande), mettre à jour l'état d'une demande, etc...
- **DELETE** : Afin de pouvoir supprimer des données
 - Supprimer une demande, etc...

Source : <https://dev.mysql.com/doc/refman/8.0/en/privileges-provided.html>

C. Connexion

Toutes les interactions avec la base de données se font depuis le service « database.js ». La connexion à la base de données se fait donc depuis le même fichier.

Avec mysql2, il y a deux possibilités pour se connecter à la base de données :

- **Créer une connexion à chaque requête** : A chaque fois qu'une requête doit être faite, une connexion vers la base de données est établie, puis fermée une fois la requête effectuée.
- **Utiliser un pool** : Lors du lancement de l'application, un pool de 10 connexions (par défaut) est créé. À chaque fois qu'une requête doit être faite, une connexion sera utilisée, puis libérée une fois la requête effectuée.

La solution du pool présente plusieurs avantages :

- Temps de réponses plus rapide
- Moins consommateur en termes de ressources

La connexion à la base de données se fait donc via un pool :

```
const pool = mysql.createPool({  
  host: process.env.DB_HOST, // Database host  
  user: process.env.DB_USER, // Database user  
  password: process.env.DB_PASSWORD, // Database user's password  
  database: process.env.DB_NAME, // Database name  
});
```

Pour chaque requête vers la base de données, une connexion doit être récupérée puis libérée une fois la requête effectuée :

```
const getData = async () => {  
  // Récupération d'une connexion  
  const connection = await pool.getConnection();  
  try {  
    // Requête à effectuer  
  } catch (error) {  
    // Traitement en cas d'erreur  
  } finally {  
    // Libération de la connexion  
    connection.release();  
  }  
};
```

D. Prepared statements

La librairie `mysql2` permet l'utilisation de prepared statements pour une exécution efficace et sécurisée des requêtes SQL. Ces derniers protègent contre les injections SQL en séparant les données de la requête. L'exécution se fait en deux étapes : la préparation, où le serveur vérifie la syntaxe et initialise les ressources, et l'exécution, où les valeurs des paramètres sont liées à la requête et envoyées pour exécution.

Pour effectuer des prepared statements avec `mysql2`, les requêtes doivent se faire de la manière suivante :

```
const query = 'INSERT INTO table (column1, column2) VALUES (?, ?)';  
const [result] = await connection.execute(sql, [data.value1, data.value2]);
```

Dans cet exemple, « ? » dans la « query » est un placeholder qui sera remplacé par les valeurs fournies dans le tableau passé en deuxième argument à la méthode `execute`. Les valeurs sont échappées automatiquement par `mysql2` pour prévenir les injections SQL.

Source : <https://sidorares.github.io/node-mysql2/docs/documentation/prepared-statements>

E. Transactions

Les transactions sont essentielles pour garantir l'intégrité des données. Une transaction est une série d'opérations qui doivent toutes réussir pour que la transaction soit réussie. Si une opération échoue, toutes les opérations sont annulées, et la base de données revient à son état précédent.

Dans le cas de la requête pour sauvegarder une demande par exemple :

- La transaction débute :

```
await connection.beginTransaction();
```

- La série d'opération est effectuée
 - o Insertion d'un nouvel enregistrement dans la table person ou la mise à jour d'un enregistrement existant
 - o Insertion d'un enregistrement dans la table demand
 - o Insertion de plusieurs enregistrements dans la table vehicle
 - o Insertion de plusieurs enregistrements dans la table occupant.
- Si toutes ces opérations réussissent, la transaction est validée. Cela signifie que toutes les modifications apportées à la base de données pendant la transaction sont permanentes.

```
await connection.commit
```

- Si une erreur se produit lors de l'une de ces opérations, la transaction est annulée, et toutes les modifications apportées durant la transaction sont annulées.

```
await connection.rollback()
```

6.9. Envoi d'emails

A. Librairie utilisée

Pour envoyer des emails avec Express, Nodemailer est la solution la plus populaire. Cette librairie permet d'utiliser divers serveurs de messagerie, d'envoyer des emails HTML, de lier des pièces jointes, etc. Nodemailer est très adopté, avec environ 3,5 millions de téléchargements par semaine, garantissant un large support communautaire et de nombreuses ressources disponibles.

Source : <https://nodemailer.com/>

B. Connexion au serveur SMTP

Toutes les fonctions liées à l'envoi d'emails se trouvent dans un service « mailSender ». La connexion au serveur SMTP se fait donc également depuis le même fichier.

Pour se connecter à la boîte mail, il faut créer un « transporter ». Ce transporter va ensuite permettre d'envoyer des messages en utilisant la fonction sendMail :

```
Transporter.sendMail(from,to,objet,text)
```

Pour créer un transporter Gmail, différentes informations sont nécessaires : Le host, le port, l'usage du SSL, le nom d'utilisateur, et le mot de passe.

La configuration du transporter est donc la suivante :

```
const transporter = nodemailer.createTransport({  
  host: process.env.MAILER_HOST, //  
  port: process.env.MAILER_PORT,  
  secure: true,  
  auth: {  
    user: process.env.MAILER_USER,  
    pass: process.env.MAILER_PASSWORD,  
  },  
});
```

Comme la boîte mail possède l'authentification à 2 facteurs activée, il est nécessaire de créer un mot de passe d'application afin de pouvoir s'y connecter.

Source : <https://www.nodemailer.com/usage/using-gmail/>

C. Internationalisation

Pour que les emails puissent être envoyés dans la langue de l'utilisateur, il est nécessaire d'également mettre en place un principe d'internationalisation dans le backend.

Toutes les traductions des emails sont stockées dans un fichier « fr.json » se trouvant dans le répertoire « mailTexts » du backend. Les traductions sont ensuite chargées à l'exécution de l'application :

- Tous les contenus fichiers présents dans le répertoire « mailTexts » sont stockés dans un objet « translations », avec un attribut « lang » permettant d'identifier les langues.

```
files.forEach((file) => {  
  const lang = path.basename(file, path.extname(file));  
  translations[lang] = require(path.join(translationsPath, file));  
});
```

Lorsqu'un email doit être envoyé, la langue de l'utilisateur est passée en paramètre de la fonction et permet de sélectionner les bonnes traductions.

- L'objet « translation » contient ensuite les traductions dans la langue de l'utilisateur

```
const translation = translations[language];
```

6.10. Traitement des erreurs

A. Client

La gestion des erreurs côté client est cruciale pour garantir une expérience utilisateur fluide et informer l'utilisateur de l'état de l'application. Chaque opération potentiellement sujette à des erreurs, comme les appels API, est enveloppée dans des blocs « try-catch » pour capturer et gérer les exceptions de manière appropriée.

Hormis les erreurs HTTP 401 (Non Autorisé) et 403 (Interdit) liées à l'authentification, qui sont traitées par les intercepteurs Axios, toutes les autres erreurs sont propagées dans les composants par les services. Cela permet de gérer tous les cas de figure directement dans les composants et d'afficher des alertes adaptées, informant l'utilisateur de la situation.

Les alertes sont affichées à l'aide du composant Vuetify prévu à cet effet. Il y'a 2 types d'alertes pour les erreurs :

- **Info** : Elles informent l'utilisateur de situations non critiques. Par exemple, s'il tente d'exporter une liste de requêtes au format CSV mais que la liste est vide.
- **Erreur** : Elles signalent des problèmes plus graves qui entravent le fonctionnement de l'application. Par exemple, en cas de problème avec la base de données lors de la récupération de la liste des demandes.

B. Serveur

La gestion des erreurs côté serveur est primordiale pour assurer la stabilité de l'application. Toutes les opérations susceptibles de générer des erreurs, telles que les interactions avec la base de données ou les services externes, sont systématiquement dans des blocs try-catch. Cette approche permet de capturer et de traiter les exceptions de manière efficace.

En dehors des erreurs générées par les middlewares, qui renvoient directement des réponses (comme les erreurs 400 pour les formulaires invalides ou les erreurs 401 et 403 pour des problèmes d'authentification), toutes les autres erreurs sont propagées jusqu'aux routes pour un traitement centralisé.

Cette centralisation est essentielle, notamment dans des situations où des actions compensatoires sont nécessaires en réponse à des erreurs d'autres services. Par exemple, si une erreur se produit lors de l'enregistrement des données dans la base de données après avoir sauvegardé des fichiers sur le serveur, il est crucial de supprimer ces fichiers pour ne pas laisser sur le serveur des fichiers non liés à des demandes.

```
if (error.databaseError) {  
  await fileManager.deleteCarRegistrations(id);  
  res.status(500).json("An error occurred while submitting the form.");  
} else {  
  res.status(500).json("An error occurred while submitting the form.");  
}
```

Pour gérer des erreurs spécifiques, comme celles associées à l'échec de l'envoi d'un email via SMTP, qui ne compromettent pas le fonctionnement global de l'application, des "appCodes" sont utilisés. Ce « appCode » permet non seulement de choisir les actions à accomplir sur le backend, mais aussi de pouvoir afficher la bonne alerte à l'utilisateur sur le frontend.

Si le cas de l'envoi d'une demande de vignette est repris. Dans le cas où la sauvegarde des fichiers, et des données dans la base de données s'est bien passée, mais que l'email de confirmation ne parvient pas à s'envoyer, les données de la demande ne doivent pas être supprimées. Néanmoins, l'alerte affichée à l'utilisateur ne doit pas faire la mention d'un email de confirmation reçu.

```
if (error.mailerError) {
  res.status(500).json({
    message: "An error occurred while sending the emails.",
    appCode: "MAILER_ERROR",
  });
} else {
  // Erreurs liées aux fichiers et à la base de données
}
```

De plus, il est important de noter que toutes les erreurs sont systématiquement journalisées.

C. Journalisation

Dans ce projet, la journalisation est faite avec Winston. Winston est une librairie de journalisation flexible et puissante pour Node.js. Elle fournit un contrôle sur le formatage des messages de logs, ainsi que sur l'endroit où ces messages sont dirigés.

La configuration se trouve dans le fichier « logger.js » du répertoire « utils » du backend :

- **Niveau de journalisation** : Les messages de niveaux « info » et supérieurs (warn, error, fatal) sont enregistrés
- **Format des logs** : Le format des logs utilise « prettyPrint » et inclut un horodatage, ce qui permet des logs compréhensibles et agréables à lire.
Exemple de log :

```
{
  "timestamp": "2024-05-22T12:34:56.789Z",
  "level": "info",
  "message": "This is an info log",
}
```


Les logs sont écrits dans 2 fichiers différents :

- Un fichier « error-%DATE%.log » pour les messages de niveaux « error » et supérieur.
- Un fichier « combined-%DATE%.log » pour les messages de tous les niveaux.

Un système de rotation journalière des fichiers de logs est également mis en place. La rotation s'effectue avec les paramètres suivants :

- **Pattern de dates** : Les fichiers de log sont datés avec un format YYYY-MM-DD, créant ainsi un nouveau fichier chaque jour.
- **Archive compressée** : Les anciens fichiers de log sont compressés pour économiser de l'espace disque.
- **Taille maximale d'un fichier** : La taille maximale d'un fichier de log est de 20 Mo. Une fois cette limite atteinte, un nouveau fichier est créé.
- **Durée de conservation** : Les fichiers de log sont conservés pendant un maximum de 14 jours. Les fichiers plus anciens que cette période sont automatiquement supprimés.

Le logger peut être utilisé dans tous les fichiers de backend :

```
logger.info('This is an info log');
```

Il est également possible d'ajouter des éléments supplémentaires, par exemple pour ajouter le message d'erreur de la base de données, ou d'un autre service :

```
logger.error('Database error', {error})
```

Source : <https://github.com/winstonjs/winston>

7. Tests de fonctionnement

La stratégie de tests définie lors de la conception a été rigoureusement appliquée. Un rapport de tests détaillant les différentes fonctionnalités a été complété progressivement et revérifié une fois l'application entièrement développée. De plus, deux utilisateurs ont testé l'application sans connaître son fonctionnement, afin d'identifier d'éventuels cas non prévus.

A. Rapport de tests

Le rapport de tests se trouve en annexes de ce document. Voici les informations importantes sur celui-ci :

- Il y'a une page par fonctionnalité
- Les tests numérotés avec un « F » concernent le frontend, ceux avec un « B » le backend
- Si le test est validé et correspond au résultat attendu, « OK » sera inscrit dans la colonne « validation ». Dans le cas contraire, « KO » sera inscrit, et la colonne « Résultat obtenu » comprendra le détail de l'erreur.
- Des détails additionnels permettant une meilleure compréhension du test et des résultats peuvent être écrits dans la colonne « Remarque »

Voici la signature du rapport de tests :

Date	Nom et prénom	Signature
23 mai 2024	Gilliand Yoan	

B. Test des utilisateurs

Bastian Turin, apprenti informaticien en 3^{ème} année

Je trouve que c'est une application simple à prendre en main et son utilisation est intuitive. La présentation est concise et l'utilisateur s'y retrouve facilement. Je verrais très bien cette application être utilisée au sein de la HES-SO//FR. Je la trouve de très bonne qualité.

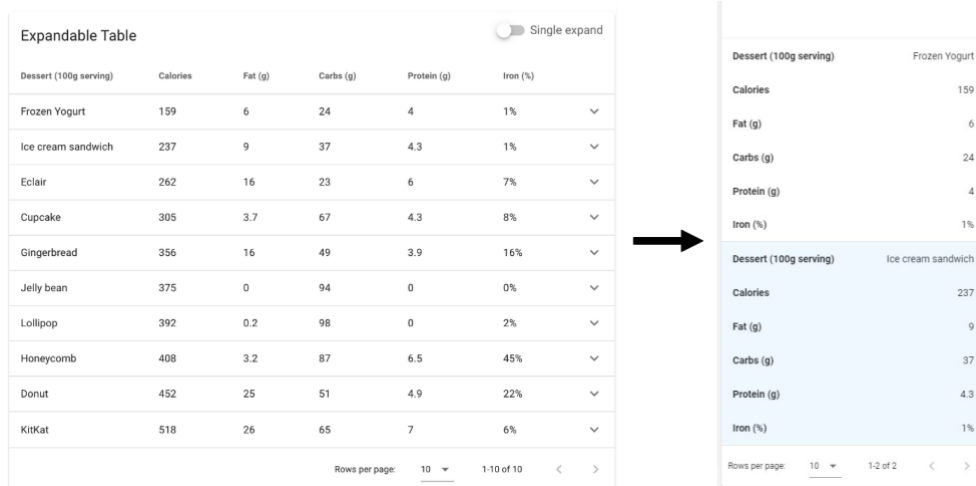
Coralie Amiet, stagiaire en préapprentissage d'informaticienne

Je trouve l'interface très intuitive, simple et propre. Cette application est facile à comprendre pour l'utilisateur et la navigation est agréable. Je la trouve très utile et je pense que cette application pourrait vraiment être utilisées au sein de nos écoles.

8. Problèmes rencontrés

A. Affichage de la liste des demandes sur mobile

Lorsque le tableau de la liste des demandes était trop large sur mobile, le tableau dépassait de la page, mais ne permettait pas à l'utilisateur de faire défiler l'écran vers la droite. Cependant, après quelques recherches, il s'est avéré que Vuetify offre un mode mobile pour les tableaux qui doit être défini manuellement.



Expandable Table Single expand

Dessert (100g serving)	Calories	Fat (g)	Carbs (g)	Protein (g)	Iron (%)
Frozen Yogurt	159	6	24	4	1%
Ice cream sandwich	237	9	37	4.3	1%
Eclair	262	16	23	6	7%
Cupcake	305	3.7	67	4.3	8%
Gingerbread	356	16	49	3.9	16%
Jelly bean	375	0	94	0	0%
Lollipop	392	0.2	98	0	2%
Honeycomb	408	3.2	87	6.5	45%
Donut	452	25	51	4.9	22%
KitKat	518	26	65	7	6%

Rows per page: 10 1-10 of 10

Dessert (100g serving)	Frozen Yogurt
Calories	159
Fat (g)	6
Carbs (g)	24
Protein (g)	4
Iron (%)	1%

Dessert (100g serving)	Ice cream sandwich
Calories	237
Fat (g)	9
Carbs (g)	37
Protein (g)	4.3
Iron (%)	1%

Rows per page: 10 1-2 of 2

Figure 34 : Tableaux vuetify en mode mobile

Source : <https://vuetifyjs.com/en/api/v-data-table/>

B. Encodage des caractères dans les fichiers CSV

Lors des premiers essais d'exportation de la liste des demandes au format CSV sur Windows, les caractères spéciaux comme le « é » n'étaient pas correctement encodés en UTF-8. Pour résoudre ce problème, le Byte Order Mark a dû être configuré dans le service du backend « fileManager.js ».

Le Byte Order Mark (BOM) est une séquence de bits placée au début d'un fichier texte pour indiquer l'ordre des octets et le type d'encodage utilisé, notamment pour les formats Unicode comme UTF-8.

Source : <https://github.com/zemirco/json2csv/issues/182>

C. Authentification sur Safari et Firefox

Lorsqu'un utilisateur tente de s'authentifier via la gateway sur Safari ou Firefox, la page d'authentification de Microsoft s'affiche en boucle. Le problème vient du fait que pour récupérer le « access_token » sur la gateway, l'application a besoin d'un cookie « connect.sid ». Comme ce cookie vient du domaine « caticapp06.isc.heia-fr.ch » et non du domaine du site, il est considéré comme un cookie de pistage et n'est pas conservé sur le navigateur. Le problème n'est donc pas résoluble, car il provient de la « CA TIC Authentication Gateway » et non pas de l'application en elle-même.

Source : <https://support.mozilla.org/fr/kb/cookies-tiers-et-protection-contre-pistage>

D. Emails

Après environ deux jours d'utilisation, la boîte mail de l'application a été supprimée par Gmail, suspectant qu'elle ait été créée par un robot. Cela est probablement dû à l'envoi massif d'emails lors des tests.

Cependant, le traitement des erreurs d'email étant géré de manière spécifique, l'application a continué de fonctionner correctement en attendant la création d'une nouvelle boîte mail. Les cas d'erreurs sont traités comme suit :

- **Envoi d'une demande de vignette** : Lorsque l'utilisateur envoie une demande, un message de confirmation indique que la demande a été enregistrée, mais sans mentionner l'envoi d'un email de confirmation, contrairement à la situation normale.
- **Acceptation ou refus d'une vignette** : Lorsque l'administrateur met à jour l'état d'une demande, il est notifié que l'email n'a pas pu être envoyé, afin qu'il puisse manuellement informer l'utilisateur de la décision.

9. Amélioration et évolution

A. Authentification

La solution actuelle utilisée pour l'authentification peut être améliorée sur deux aspects :

1. Il est essentiel de trouver une solution permettant la connexion via les navigateurs Firefox et Safari, utilisés par environ 15 % des personnes. Les contraindre à utiliser un autre navigateur spécifiquement pour eParking est peu pratique et nuit à leur expérience.
2. Avec la version actuelle de la « CA TIC Authentication Gateway », il n'est pas possible d'identifier spécifiquement les utilisateurs de la HES-SO//FR. Tous les utilisateurs du tenant HES-SO peuvent s'authentifier.

Source : <https://gs.statcounter.com/browser-market-share/desktop/worldwide>

B. Gestion des prix et des secteurs depuis l'application

Actuellement, les prix et les secteurs ne peuvent être gérés que depuis la base de données. Pour simplifier la tâche des administrateurs, il serait utile de leur offrir la possibilité de les gérer directement depuis l'interface d'administration de l'application.

C. Onglet personnel pour les utilisateurs

Afin de rendre l'expérience utilisateur plus plaisante, il pourrait être bien de faire en sorte qu'ils aient un onglet personnel, ou ceux-ci peuvent voir l'historique de leur demandes par exemple.

10. Conclusions

A. Technique

L'application eParking est fonctionnelle. Toutes les fonctionnalités requises dans le cahier des charges ont été implémentées avec succès. Les utilisateurs peuvent soumettre leurs demandes de parking, et les administrateurs ont la capacité de consulter, gérer et exporter ces demandes. Cependant, l'authentification via la « CA TIC Authentication Gateway » présente certaines contraintes qui peuvent légèrement nuire à l'expérience utilisateur. En dehors de cela, toutes les fonctionnalités sont opérationnelles, et l'application est dotée d'une interface simple et facile à utiliser. Sa conception et son implémentation ont été réalisées en gardant à l'esprit la possibilité d'évolutions futures.

B. Personnelle

Travailler sur le projet eParking a été une expérience extrêmement enrichissante. J'ai pu démontrer mes compétences en développement et en gestion de projet, même sans avoir une très grande expérience en matière d'analyse et de planification. Je suis très satisfait du résultat final. Voir l'application fonctionner et savoir qu'elle pourrait être utilisée en production est très gratifiant.

Je suis particulièrement fier d'avoir mené ce projet de A à Z. Cela m'a permis d'apprendre énormément, tant sur le plan technique que sur la gestion des imprévus. Les défis techniques rencontrés ont renforcé mes compétences en résolution de problèmes et en gestion de projet.

Si je devais refaire ce projet, je prioriserais les aspects les plus complexes dès le début pour mieux anticiper et résoudre les problèmes techniques. Cela m'aurait probablement permis de gagner du temps et d'améliorer encore la qualité de l'application.

En résumé, cette expérience m'a permis de renforcer mes compétences, d'apprendre de nouvelles technologies et de livrer une application fonctionnelle et utile. Je suis confiant dans le potentiel de cette application et heureux de savoir que mon travail contribuera à améliorer l'efficacité et la facilité de gestion des demandes de parking.

11. Remerciements

Je tiens à adresser des remerciements à mon responsable, M. Joël Dacomo, pour la formation et l'accompagnement qu'il m'a fournis tout au long de ces quatre années d'apprentissage. Ses conseils avisés et ses remarques constructives m'ont continuellement poussé à me remettre en question, à évaluer mes choix et à approfondir mes réflexions afin de produire les meilleurs travaux possibles.

Je remercie également M. Jean-Pierre Gauch et M. Nicolas Rouiller pour leur encadrement, leurs conseils et leurs idées lors de leurs deux visites à la Haute École d'Ingénierie et d'Architecture de Fribourg.

Enfin, j'adresse mes remerciements à mes collègues et ex-collègues apprentis, toujours disponibles et prêts à m'aider en cas de besoin.

12. Déclaration de non plagiat

Je soussigné Yoan Gilliand déclare que les informations contenues dans ce rapport de travail pratique individuel rendu ce jour le 23.05.2024 dans le cadre de la procédure de qualification de mon CFC d'informaticien, ne sont pas plagiées. Toutes les informations de sources extérieures ainsi que les informations fournies par des tiers durant le déroulement du travail sont consignées.

Date	Nom et prénom	Signature
23 mai 2024	Gilliand Yoan	

13. Sources et références

Sources et références
https://api-ti.pages.forge.hefr.ch/
https://auth-gateway.pages.forge.hefr.ch/ca-tic-authentication-gateway/deployment/
https://axios-http.com/
https://dataedo.com/download/AdventureWorks.pdf
https://developer.mozilla.org/
https://dev.mysql.com/doc
https://eslint.org/
https://expressjs.com/
https://fr.freepik.com/
https://fr.wikipedia.org/wiki/Material_Design
https://github.com/zemirco/json2csv/issues/182
https://gs.statcounter.com/browser-market-share/desktop/worldwide
https://jetbrains.com/
https://medium.com/
https://nodejs.org/
https://nodemailer.com/
https://npmtrends.com/mysql-vs-mysql2
https://opswat.com
https://pinia.vuejs.org/
https://prettier.io/
https://restfulapi.net
https://router.vuejs.org/
https://sidorares.github.io/node-mysql2/docs/documentation/prepared-statements
https://sinfo.hefr.ch/

https://stackoverflow.com/questions/
https://vitejs.dev/guide/env-and-mode
https://vuejs.org
https://vuetifyjs.com/
https://www.flaticon.com/fr/
https://www.hefr.ch/media/xk4gwtl5/hes-so-fr_organigramme f 2024-01.pdf
https://www.npmjs.com/package/dotenv
https://www.npmjs.com/package/uuid

14. Glossaire

Glossaire	
Abréviation	Description
API	Application Programming Interface
CA TIC	Centre d'Apprentissage TIC
CFC	Certificat Fédéral de Capacité
EMF	École des Métiers de Fribourg
HEDS	Haute École de De Santé
HEG	Haute École de Gestion
HEIA-FR	Haute École d'Ingénierie et d'Architecture de Fribourg
HES-SO//FR	Hautes Écoles de Suisse Occidentale, Fribourg
HETS	Haute École de Travail Social
HTTP(-S)	Hypertext Transfer Protocol (-Secure)
ISO	International Organization for Standardization
MFA	Multi-Factor Authentication
RFC	Request For Comments
RWD	Responsive Web Design
SMTP	Simple Mail Transfer Protocol)
SQL	Structured Query Language
UUID	Universally Unique Identifier
VCS	Version Control System

15. Annexes

L'annexe « TPI_2024_149380_GilliandYoan » est structurée comme suit :

Annexe	Description
CDC	Cahier des charges au format PDF
Code	Contient le code source de l'application
DocumentationTechnique	Documentation technique du projet
Journal	Journal de bord du projet au format PDF
Mockups	Contient les différents mockups au format PDF
Planification	Planification du projet
PV	Contient les différents PV des séances au format PDF
Séquences	Contient les UML de séquences au format PNG
Tests	Rapport de tests au format XLSX

16. Table des illustrations

Figure 1 : Planification - Du 7 au 15 mai.....	7
Figure 2 : Planification - Du 16 au 23 mai.....	7
Figure 3 : Principe de fonctionnement	10
Figure 4 : Diagramme de cas d'utilisation.....	11
Figure 5 : Ébauche conceptuelle - Authentification	22
Figure 6 : Ébauche conceptuelle - Envoi d'une demande pour obtenir une vignette	23
Figure 7 : Ébauche conceptuelle - Affichage de la liste des demandes	23
Figure 8 : Ébauche conceptuelle - Téléchargement d'une carte grise.....	23
Figure 9 : Ébauche conceptuelle - Exportation de la liste des demandes	24
Figure 10 : Ébauche conceptuelle - Affichage des détails d'une demande	24
Figure 11 : Ébauche conceptuelle - Acceptation ou refus d'une demande	25
Figure 12 : Ébauche conceptuelle - Suppression d'une demande	25
Figure 13 : HTTP vs HTTPS	26
Figure 14 : Architecture 3-tiers.....	28
Figure 15 : Architecture globale	28
Figure 16 : Diagramme de séquences - Authentification	29
Figure 17 : Diagramme de séquences - Envoi d'une demande pour obtenir une vignette	30
Figure 18 : Diagramme de séquences - Affichage de la liste des demandes	31
Figure 19 : Diagramme de séquences - Exportation de la liste des demandes	32
Figure 20 : Diagramme de séquences - Affichage des détails d'une demande	33
Figure 21 : Diagramme de séquences - Téléchargement d'une carte grise	34
Figure 22 : Diagramme de séquences - Acceptation ou refus d'une demande.....	35
Figure 23 : Diagramme de séquences - Suppression d'une demande.....	36
Figure 24 : Base de données.....	37
Figure 25 : Mockup - Formulaire - Format ordinateur	43
Figure 26 : Mockup - Liste des demandes - Format ordinateur	44
Figure 27 : Mockup - Détails d'une demande - Format ordinateur.....	45
Figure 28 : Mockups - Format mobile	46
Figure 29 : Base de données - CA TIC Authentication Gateway.....	50
Figure 30 : Diagramme de flux - authenticateUser.....	53
Figure 31 : Intercepteurs Axios	54
Figure 32 : Middlewares.....	55
Figure 33 : Validation des données du formulaire - Frontend	57
Figure 34 : Tableaux vuetify en mode mobile.....	67